

Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación



IIC2613 - Inteligencia Artificial

Redes Neuronales Convolucionales (CNN)

Hans Löbel

Dpto. Ingeniería de Transporte y Logística
Dpto. Ciencia de la Computación

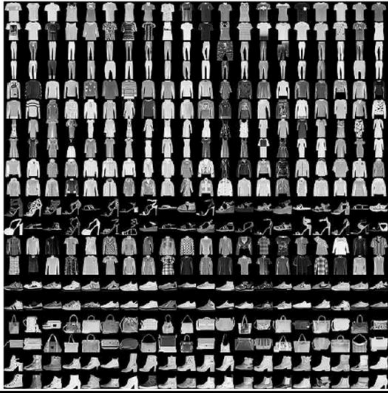
Antes de empezar

- Está publicada una guía de ejercicios para la I2, hecha con ejercicios de interrogaciones antiguas.
- La tasa de respuesta de la Encuesta de Evaluación Docente está, hasta ahora, muy baja:

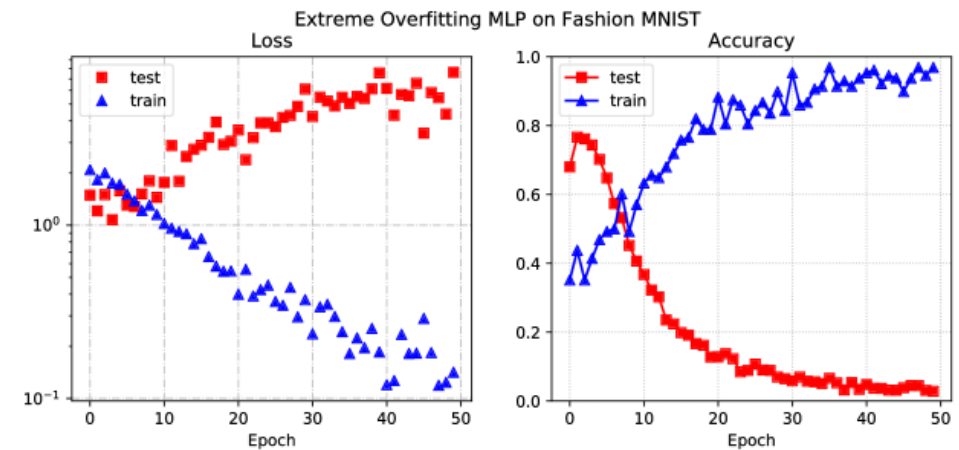
Código	Profesor	Inscripciones	Respondido	Response Rate
IIC2613-1	Hans Albert Lobel Diaz	153	5	3,27%

CNNs buscan responder la siguiente pregunta: ¿necesitamos redes distintas a un MLP para datos visuales?

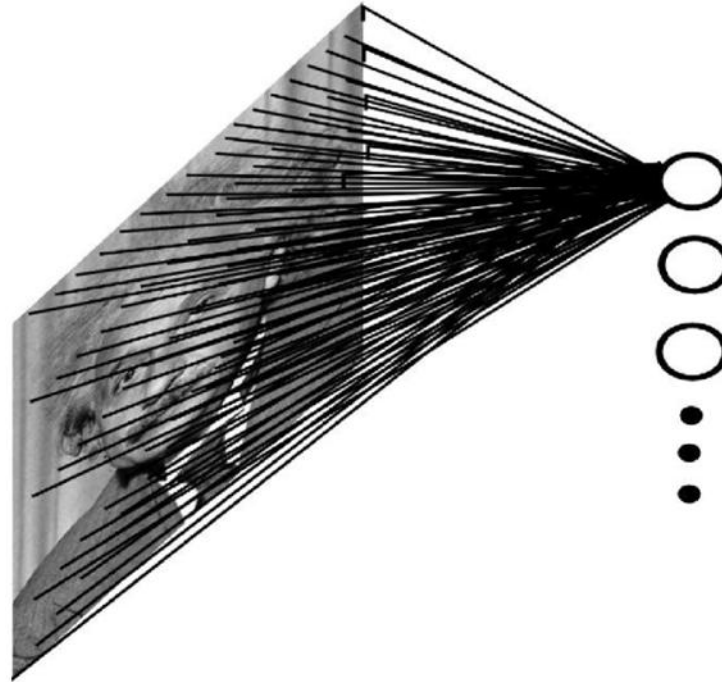
Tal como vimos anteriormente, los MLP fracasan catastróficamente en conjuntos de datos visuales más difíciles que MNIST.

Label	Description	Examples
0	T-Shirt/Top	
1	Trouser	
2	Pullover	
3	Dress	
4	Coat	
5	Sandals	
6	Shirt	
7	Sneaker	
8	Bag	
9	Ankle boots	

¿Por qué pasa esto?



Un primer culpable es la excesiva (ridícula) cantidad de parámetros



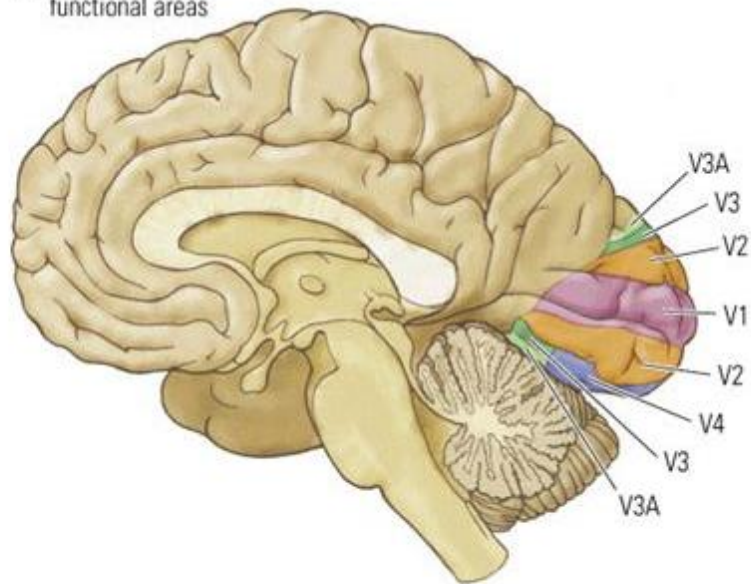
- Imagen de 256x256
- Capa *fully connected* de 256 neuronas
- 16.777.216 parámetros

Cada neurona ve todo simultáneamente, ¿es esto adecuado para las imágenes?

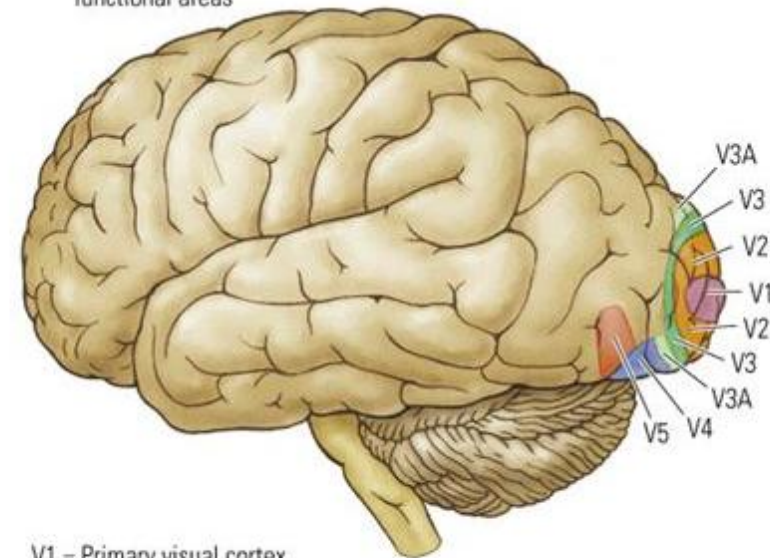
Veamos un poco qué pasa en la corteza visual de los mamíferos

Areas V1 – V5

(A) Medial view of functional areas

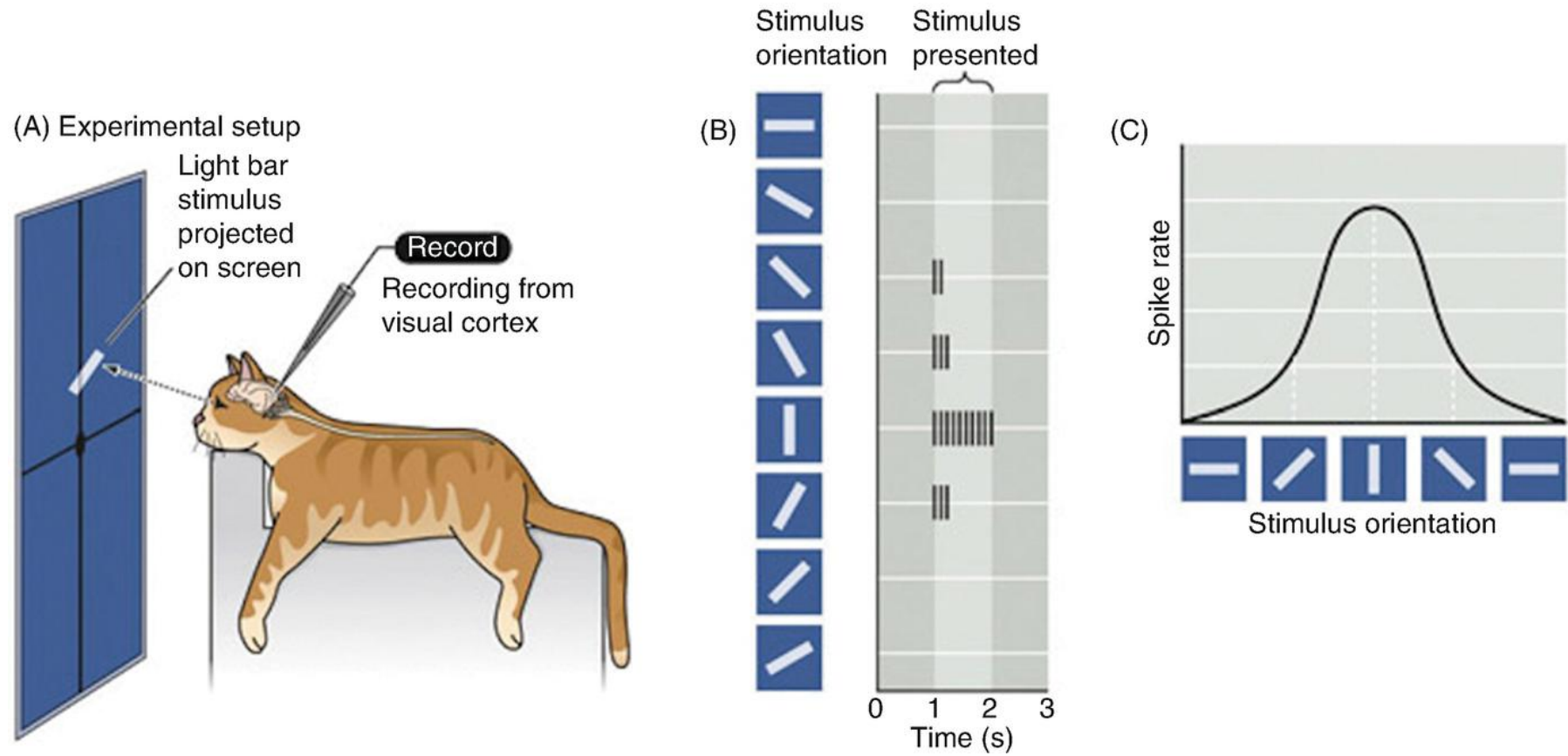


(B) Lateral view of functional areas



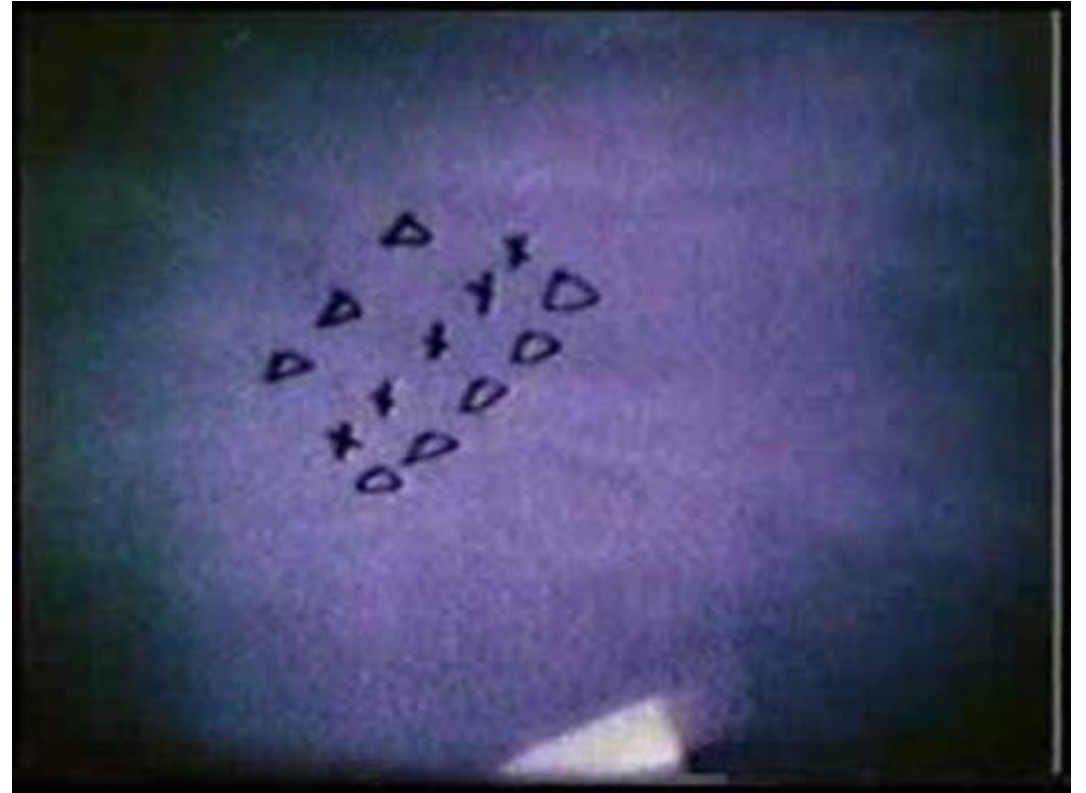
V1 = Primary visual cortex
V2–V5 = Extrastriate cortex

Patrones de estímulo son detectados por células (neuronas) específicas, altamente localizadas (Hubel y Wiesel, ≈60s)



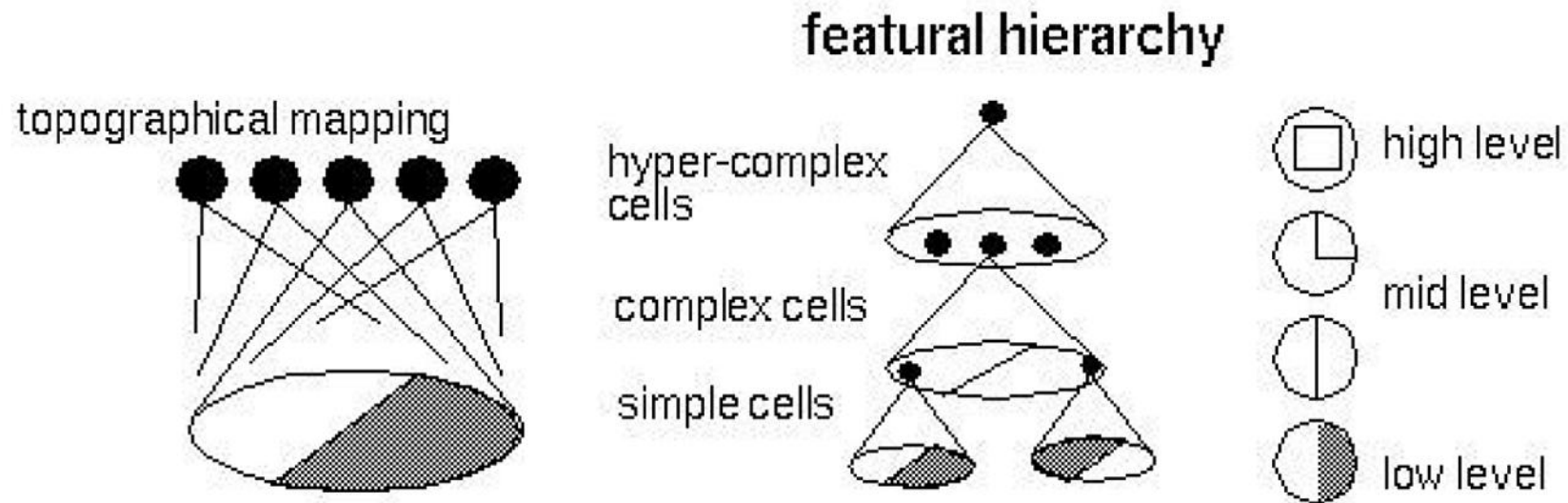


youtu.be/IOHayh06LJ4

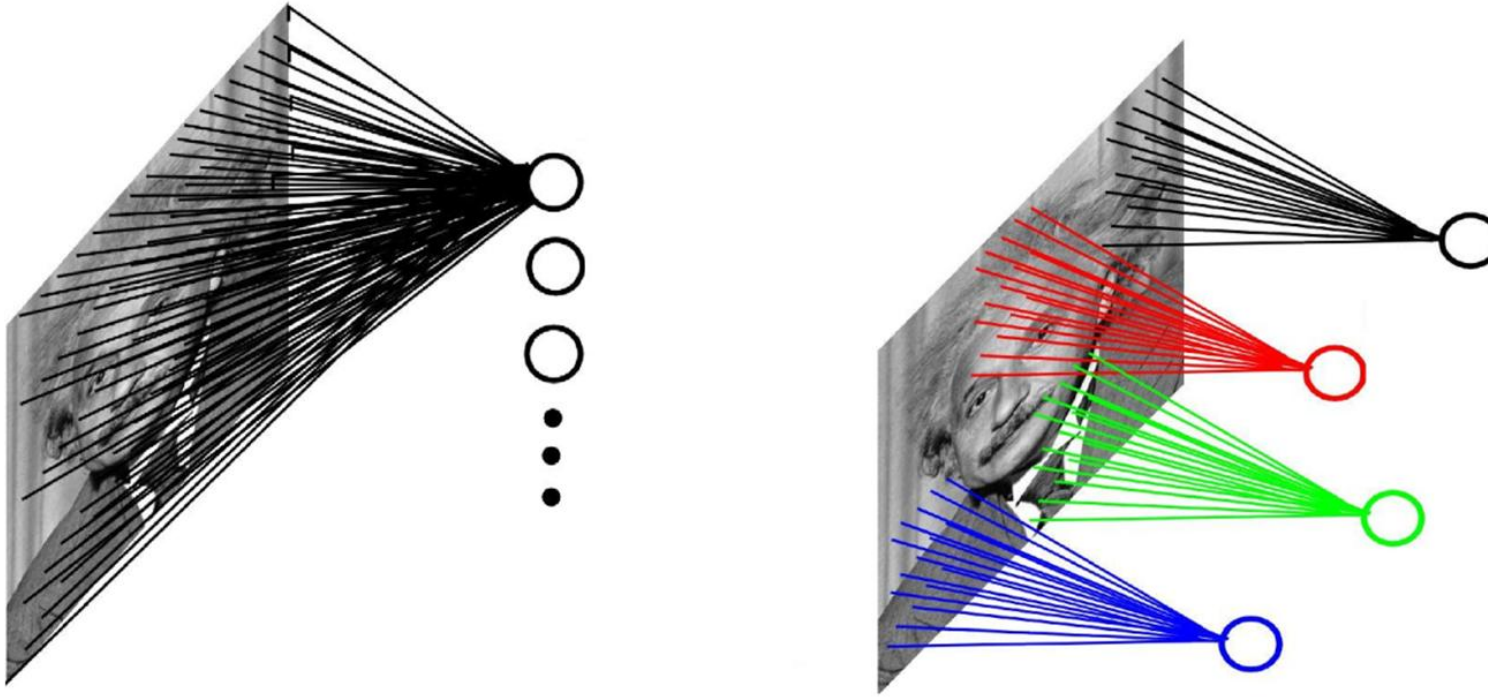


youtu.be/Cw5PKV9Rj3o

Neuronas se organizan de manera **jerárquica**, aumentando el nivel de **abstracción** de los patrones **locales** detectados



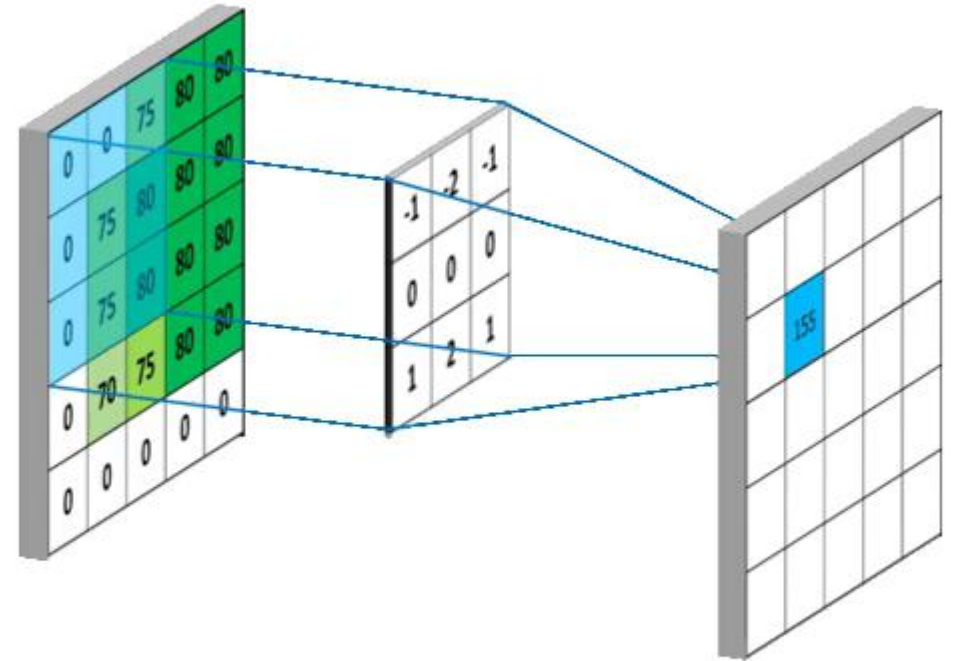
Podemos modelar este comportamiento con “neuronas deslizantes”



- Neuronas capaces de barrer la entrada proveen una reducción significativa en el número de parámetros (no depende de la conectividad)
- Además, dado su tamaño, permiten capturar patrones locales, pudiendo activarse en cualquier posición de la imagen de entrada.

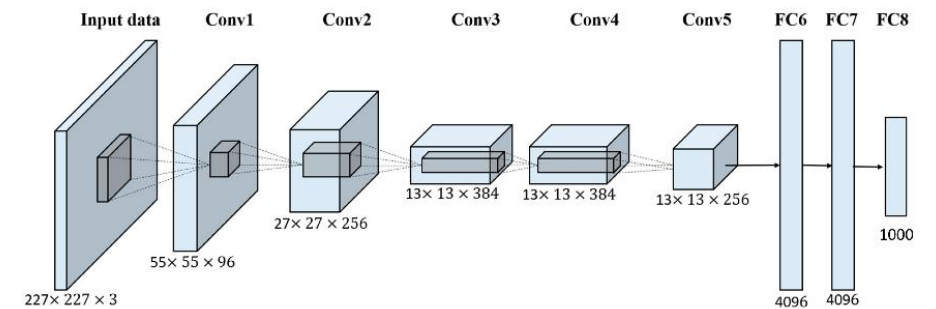
Una posible opción para modelar esto nos lo entregan las convoluciones

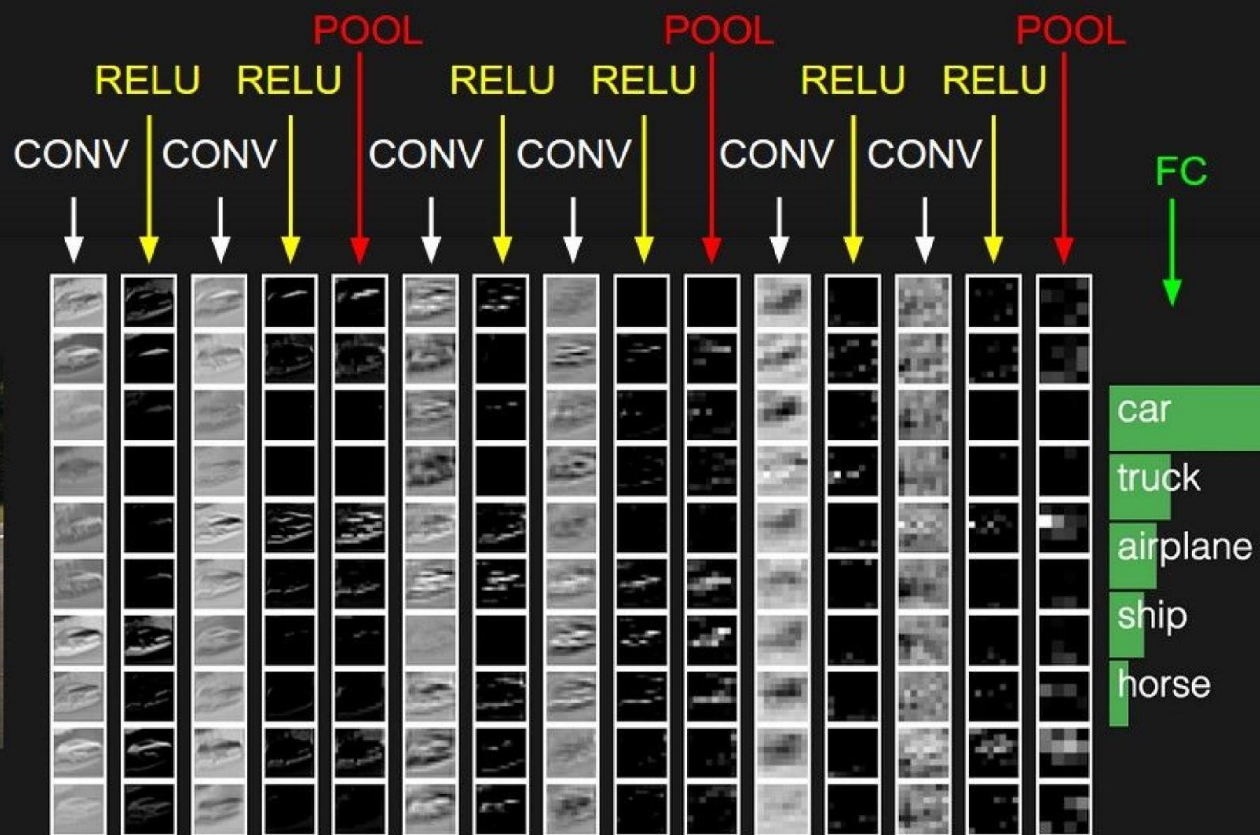
- Un patrón local puede ocurrir en cualquier parte de la imagen, pero las neuronas tradicionales no permiten capturar eso.
- La convolución entrega una solución a este problema, al modelar filtros (neuronas) locales que son aplicadas en toda la imagen.
- Los coeficientes de estos filtros son el análogo a los pesos de las neuronas en un MLP.
- Salida de una neurona ahora entrega un mapa (antes era un número), que indica dónde se podría encontrar la *feature* capturada por la neurona.



Todo esto da paso a las **redes convolucionales**, las redes profundas más exitosas/reconocidas/comunes

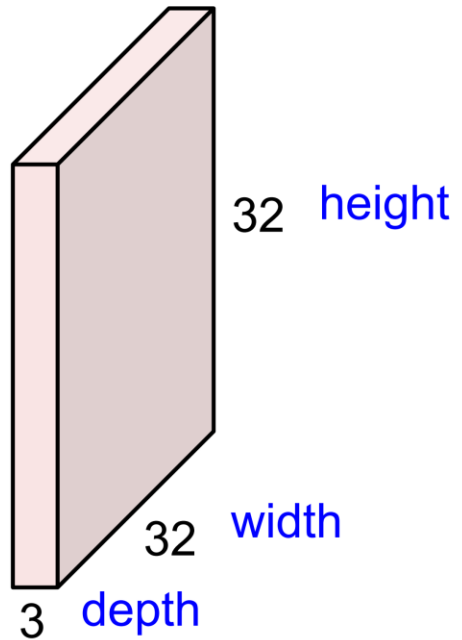
- Están formadas (generalmente) por capas convolucionales, de *pooling* y fully connected (las mismas de un MLP).
- Sustituyen la mayoría de las capas *fully connected* de los MLP, por capas convolucionales.
- Disminuyen drásticamente el número de parámetros, en comparación con los MLP.
- Sustentadas (ahora sí) en bases biológicas.





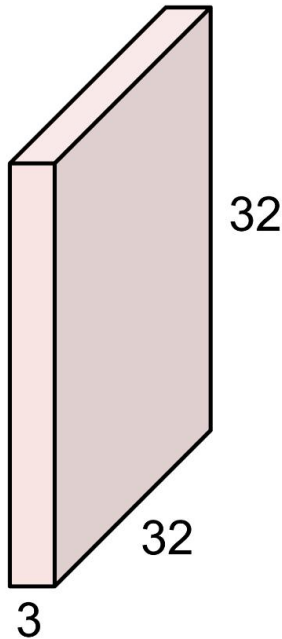
Convolution Layer

32x32x3 image



Convolution Layer

32x32x3 image



5x5x3 filter



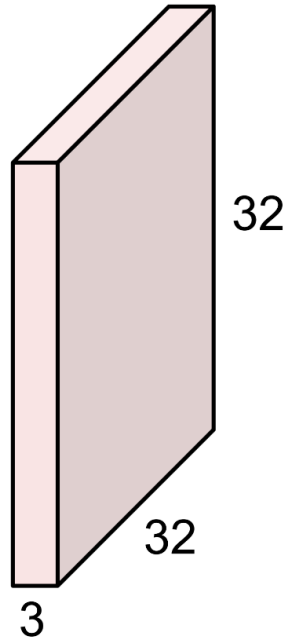
Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

$$f[x,y] * g[x,y] = \sum_{n_1=-\infty}^{\infty} \sum_{n_2=-\infty}^{\infty} f[n_1,n_2] \cdot g[x-n_1,y-n_2]$$

↑
elementwise multiplication and sum of
a filter and the signal (image)

Convolution Layer

32x32x3 image



Filters always extend the full depth of the input volume

5x5x3 filter

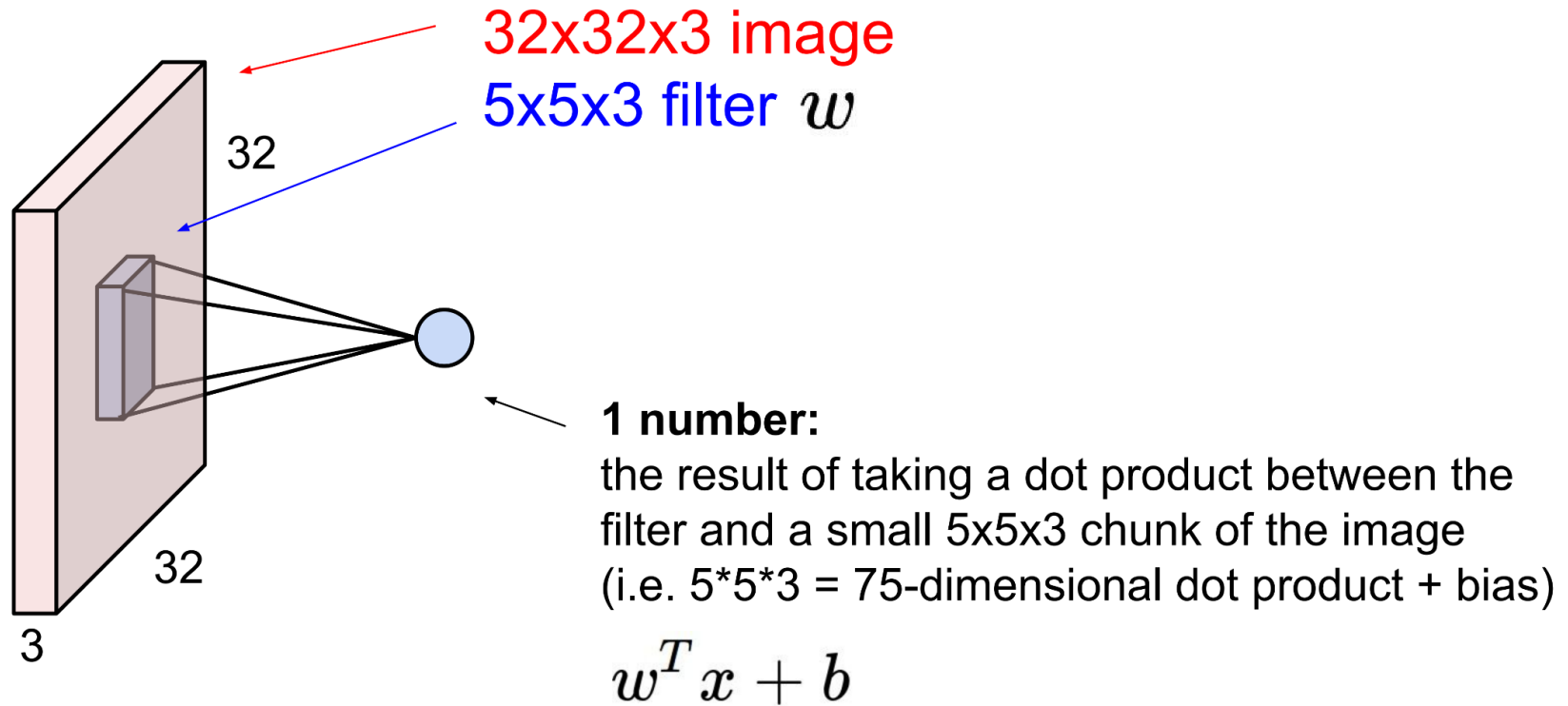


Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

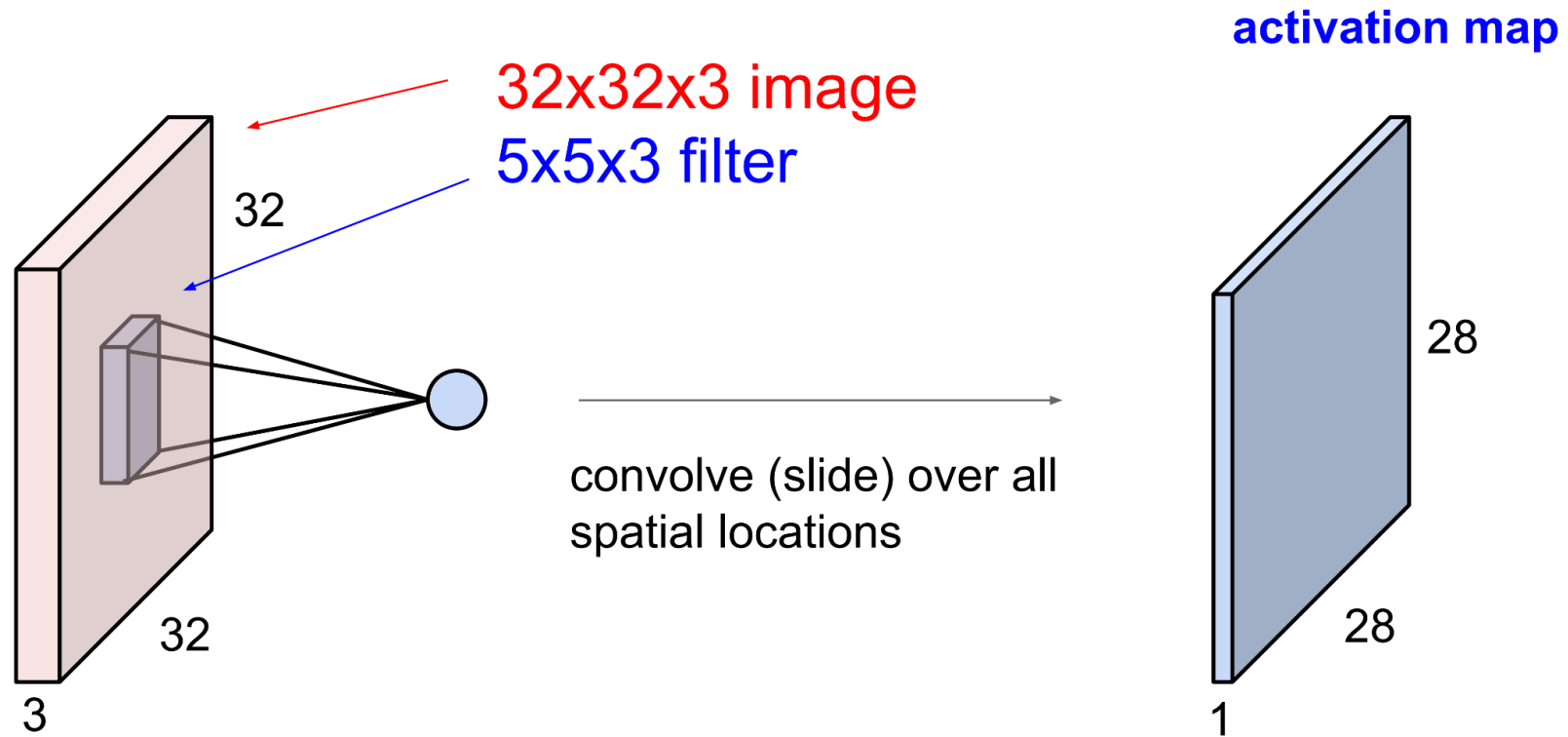
$$f[x,y] * g[x,y] = \sum_{n_1=-\infty}^{\infty} \sum_{n_2=-\infty}^{\infty} f[n_1,n_2] \cdot g[x-n_1,y-n_2]$$

↑
elementwise multiplication and sum of
a filter and the signal (image)

Convolution Layer

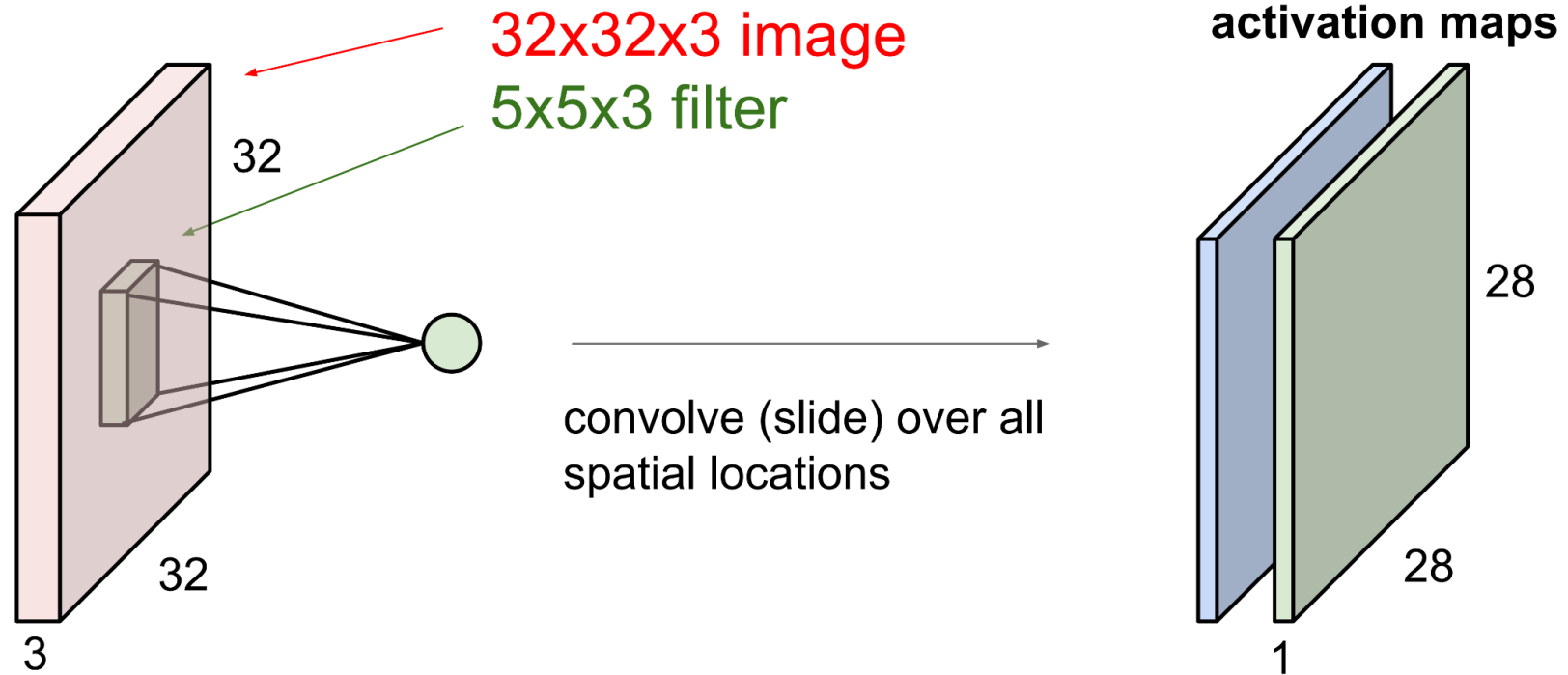


Convolution Layer

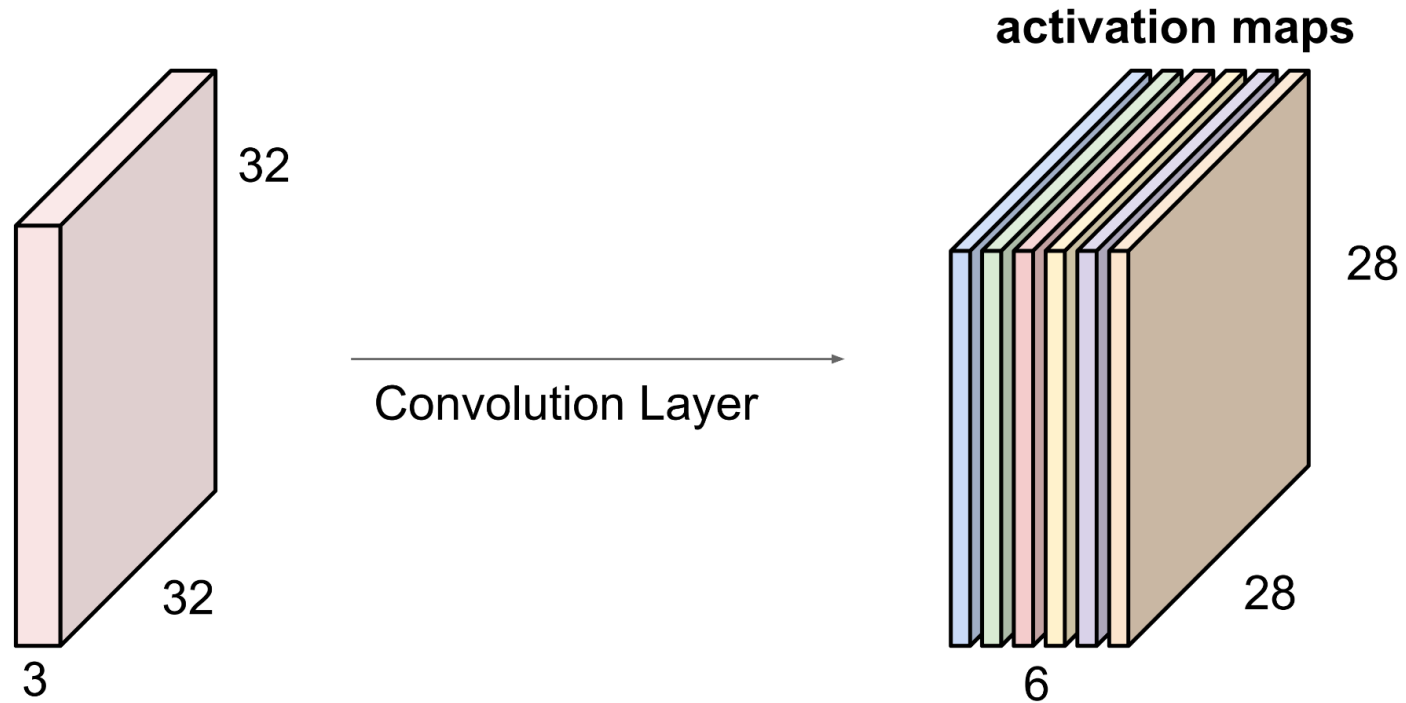


Convolution Layer

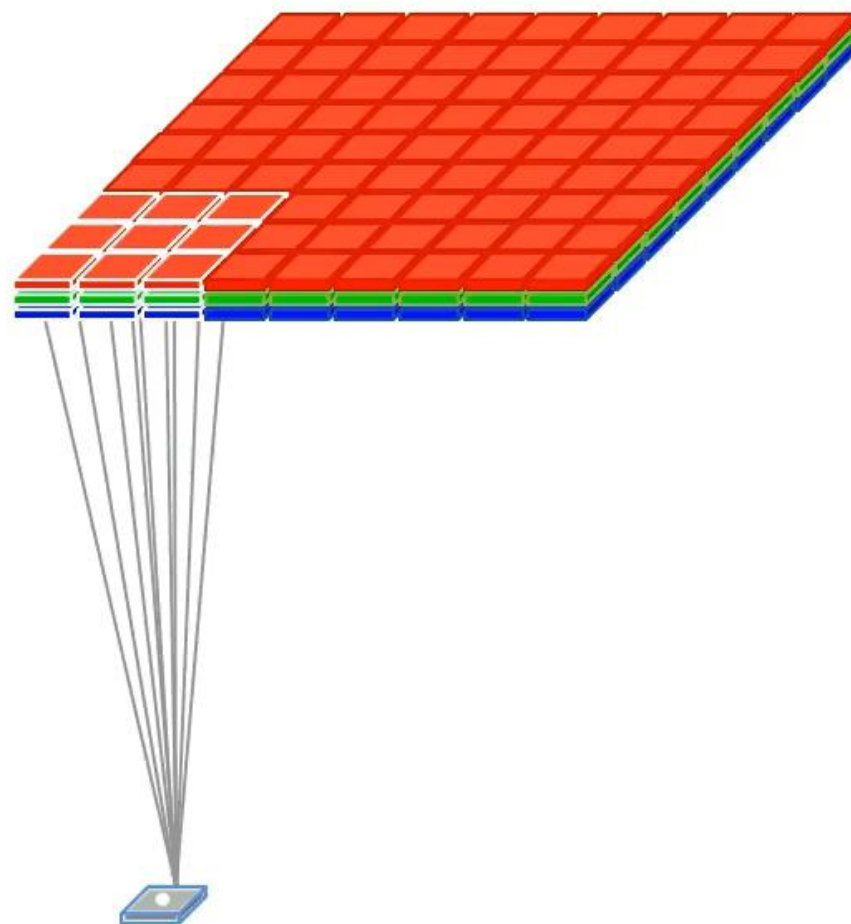
consider a second, **green** filter

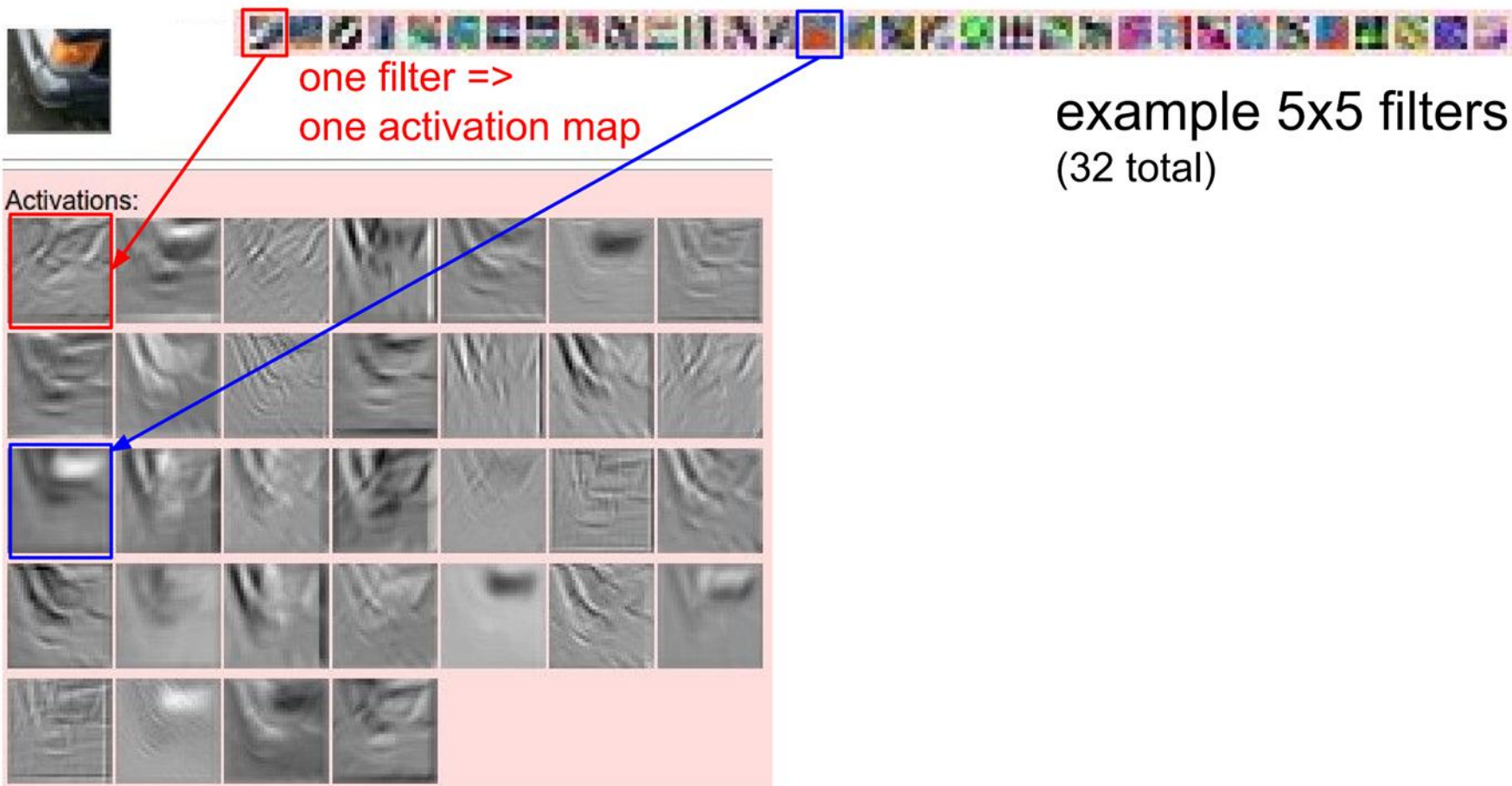


For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:



We stack these up to get a “new image” of size 28x28x6!

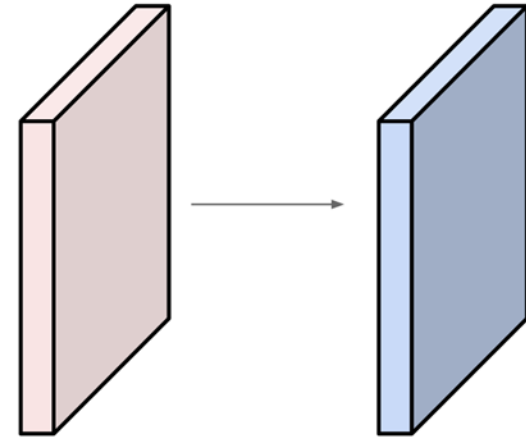




Examples time:

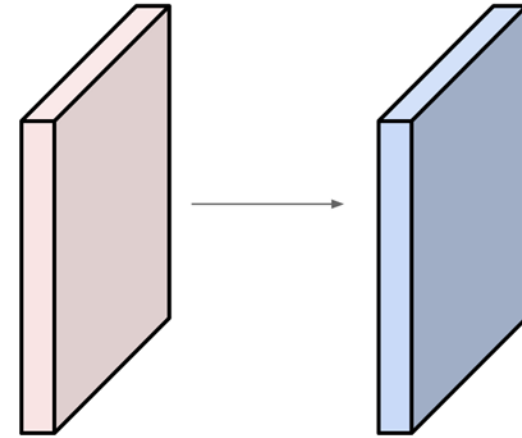
Input volume: **32x32x3**
10 5x5 filters

Number of parameters in this layer?



Examples time:

Input volume: **32x32x3**
10 **5x5** filters

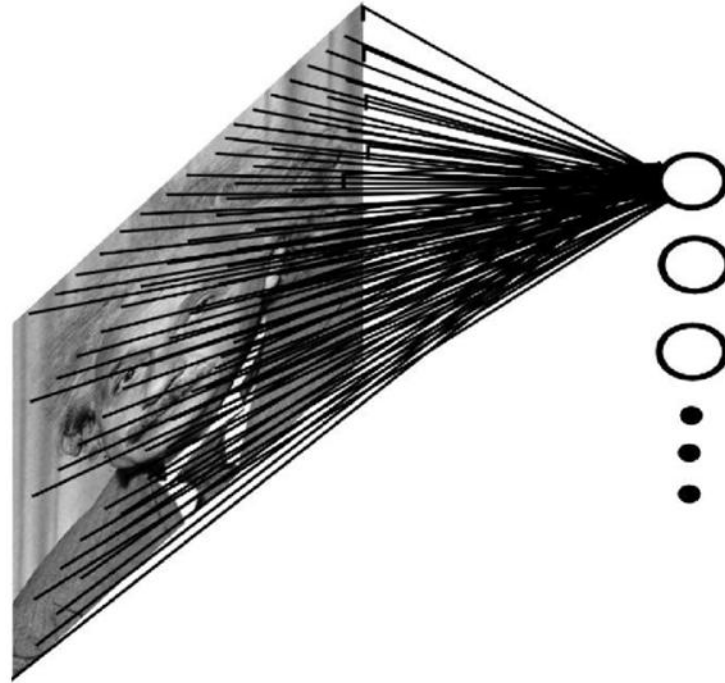


Number of parameters in this layer?

each filter has $5*5*3 + 1 = 76$ params (+1 for bias)

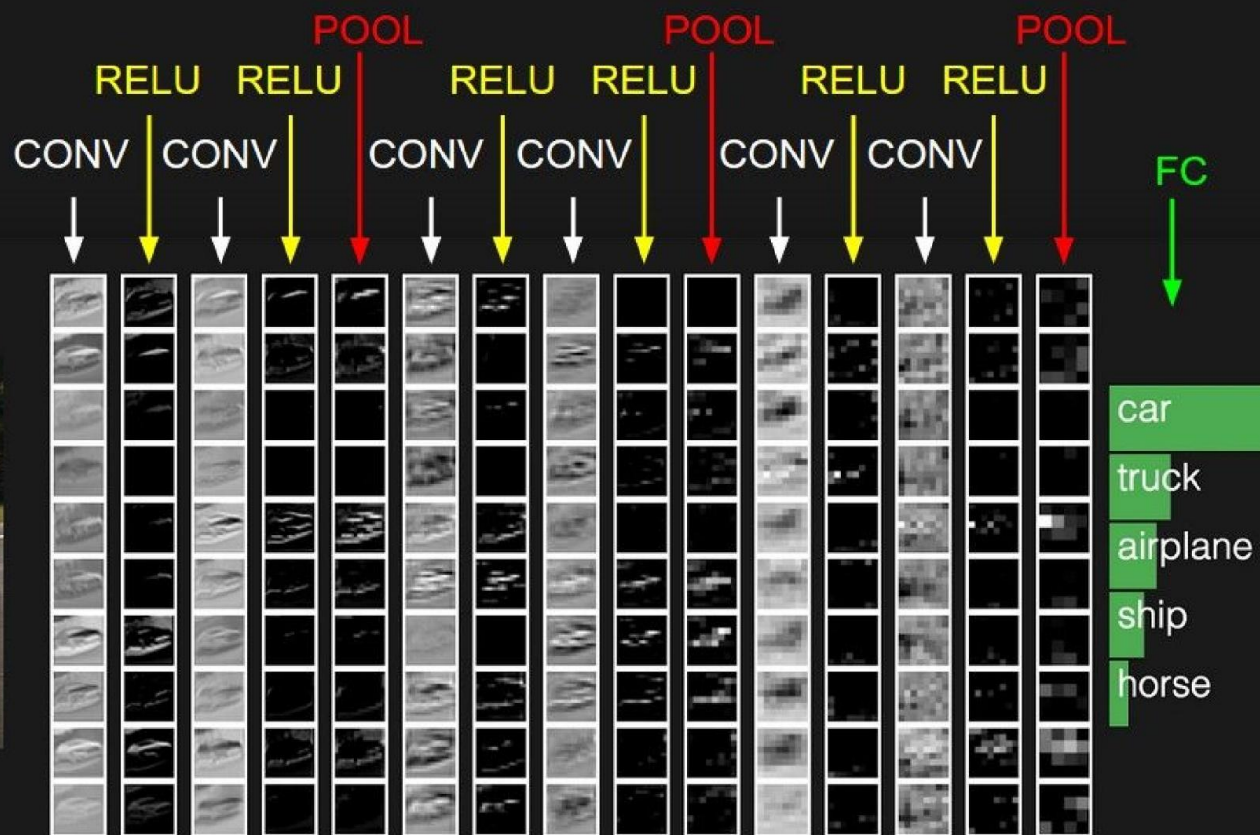
=> $76*10 = 760$

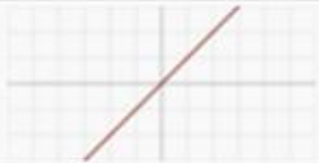
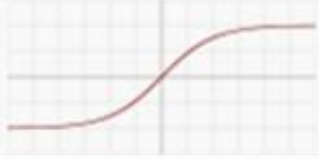
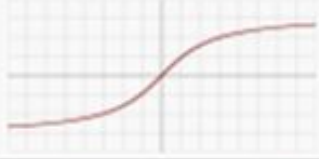
Comparemos el número anterior, con el número de parámetros de una capa *fully connected* para un caso similar

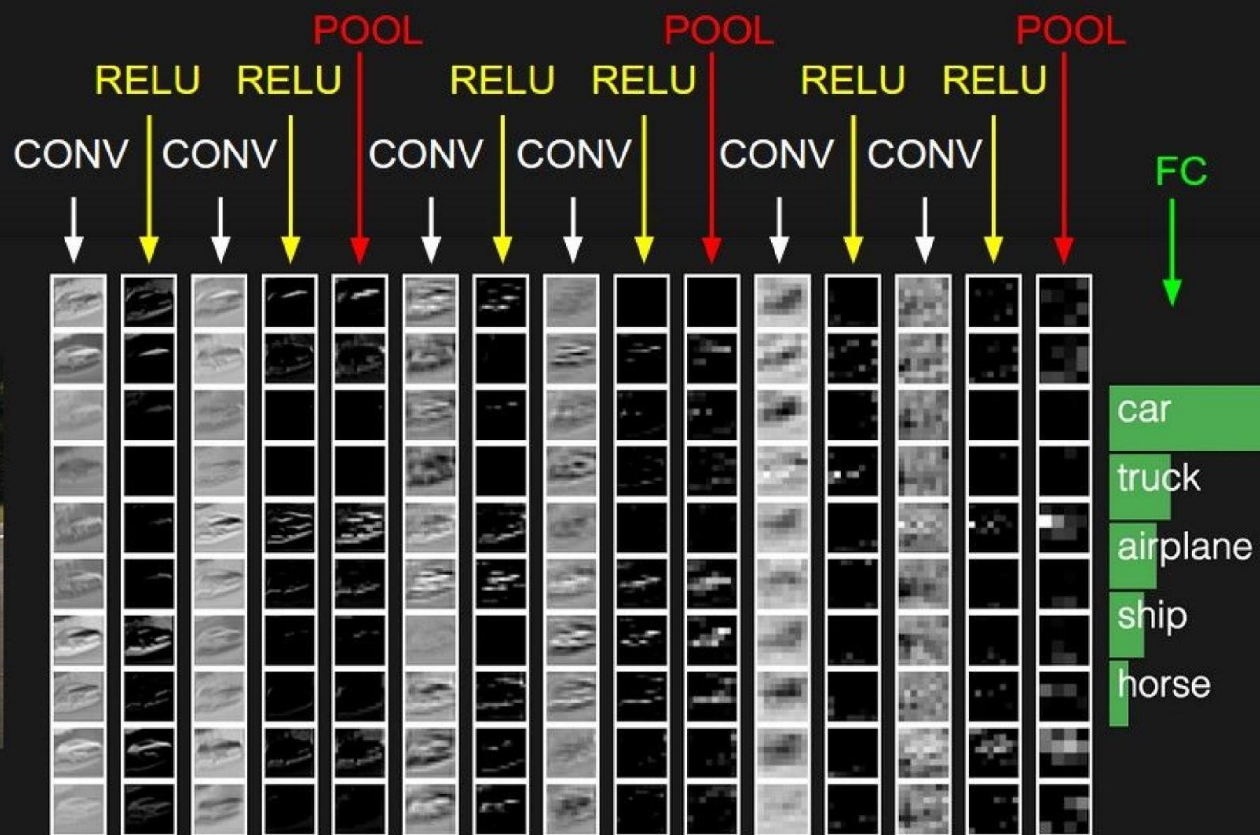


- Imagen de 32x32
- Capa *fully connected* de 10 neuronas con bias
- 10.250 parámetros

La situación es en realidad peor aún para los MLP, ya que incluso si usamos una imagen más grande como entrada (p. ej. 256x256), el número de parámetros de una red convolucional no cambiaría, pero el de un MLP se elevaría por sobre los el medio millón.

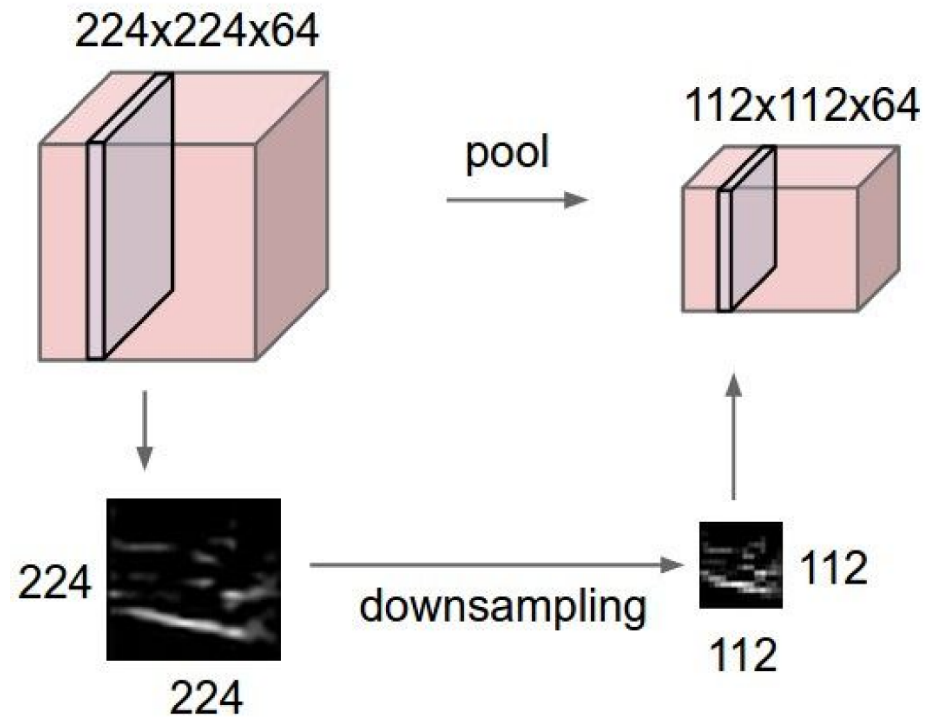


Name	Plot	Equation
Identity		$f(x) = x$
Logistic (a.k.a Soft step)		$f(x) = \frac{1}{1 + e^{-x}}$
TanH		$f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1$
ArcTan		$f(x) = \tan^{-1}(x)$
Rectified Linear Unit (ReLU)		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$

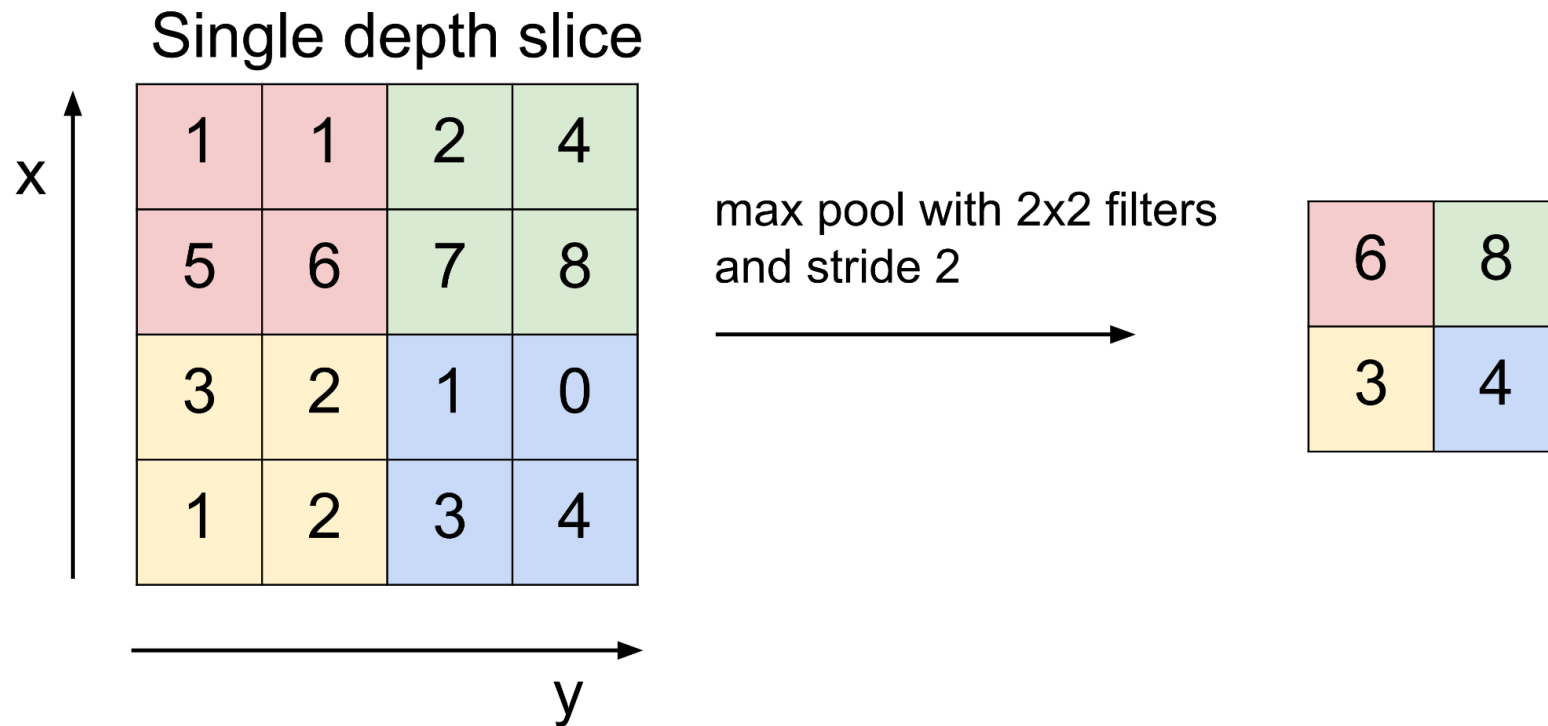


Pooling layer

- makes the representations smaller and more manageable
- operates over each activation map independently:

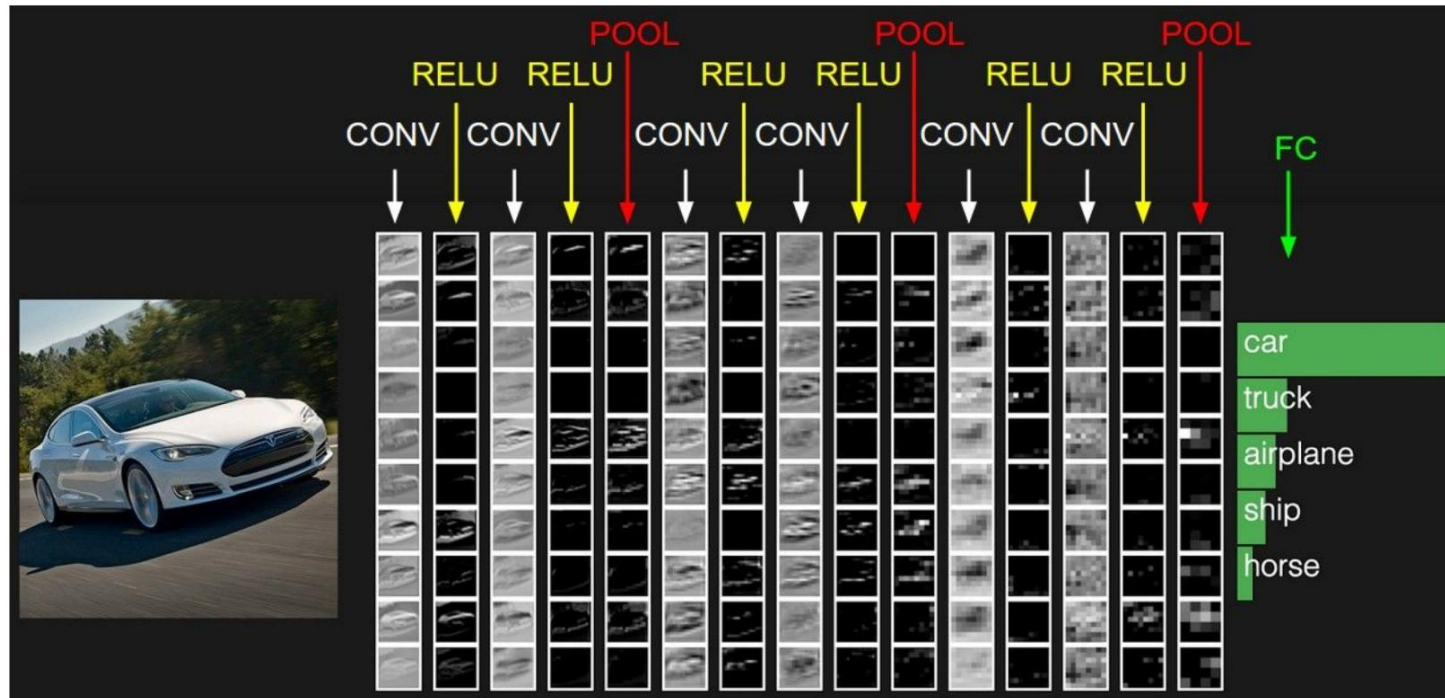


MAX POOLING

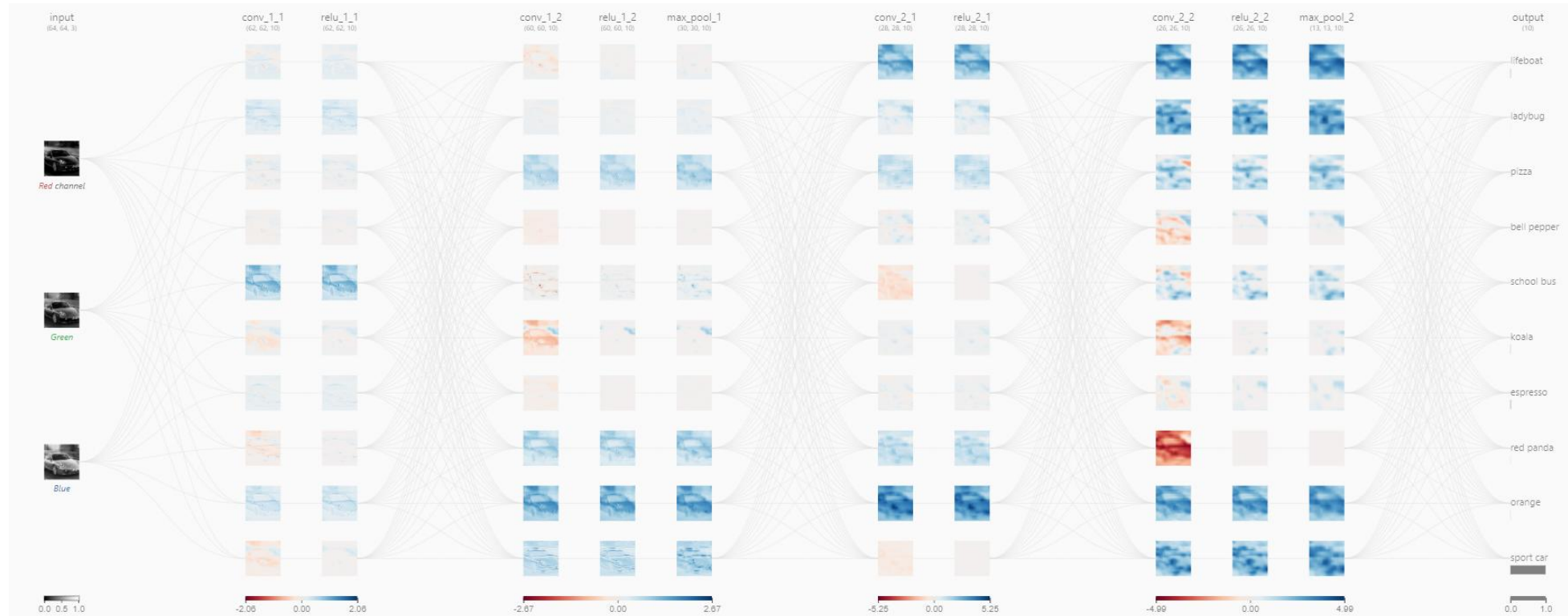


Fully Connected Layer (FC layer)

- Contains neurons that connect to the entire input volume, as in ordinary Neural Networks

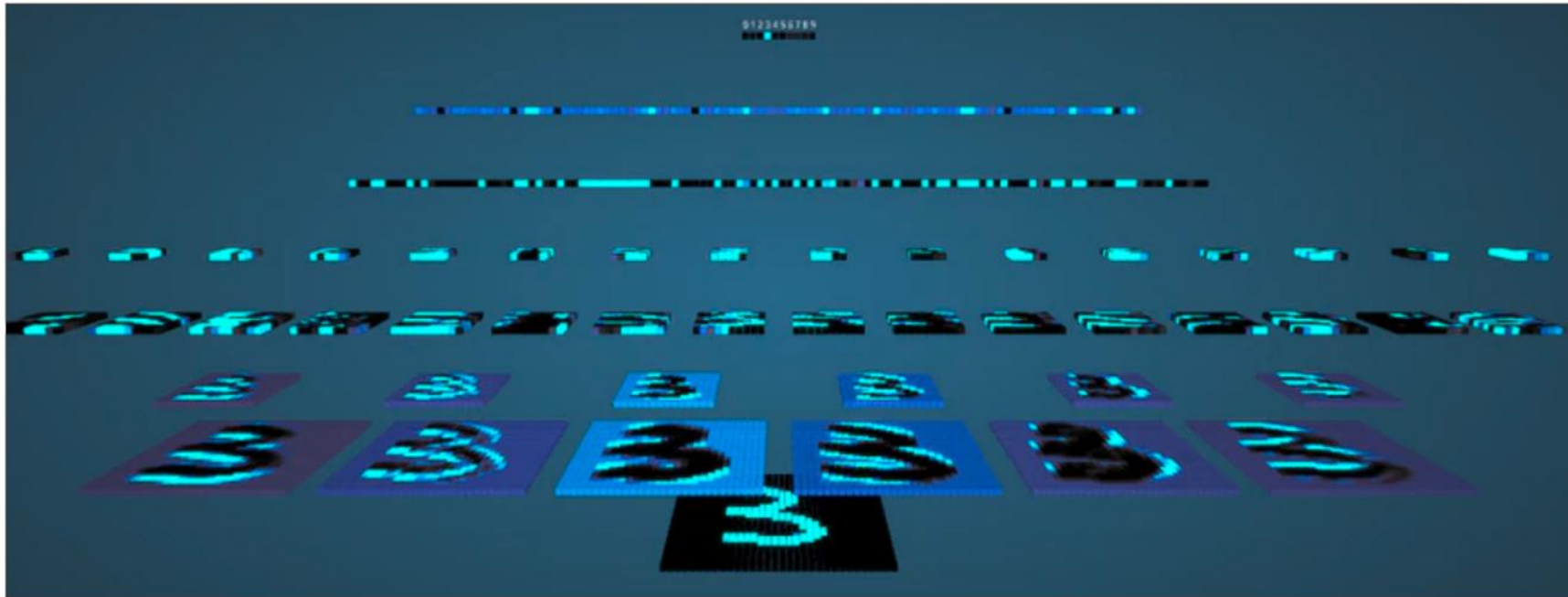


Revisemos como se **ve** todo esto en la práctica



<https://poloclub.github.io/cnn-explainer/>

Revisemos como se **ve** todo esto en la práctica



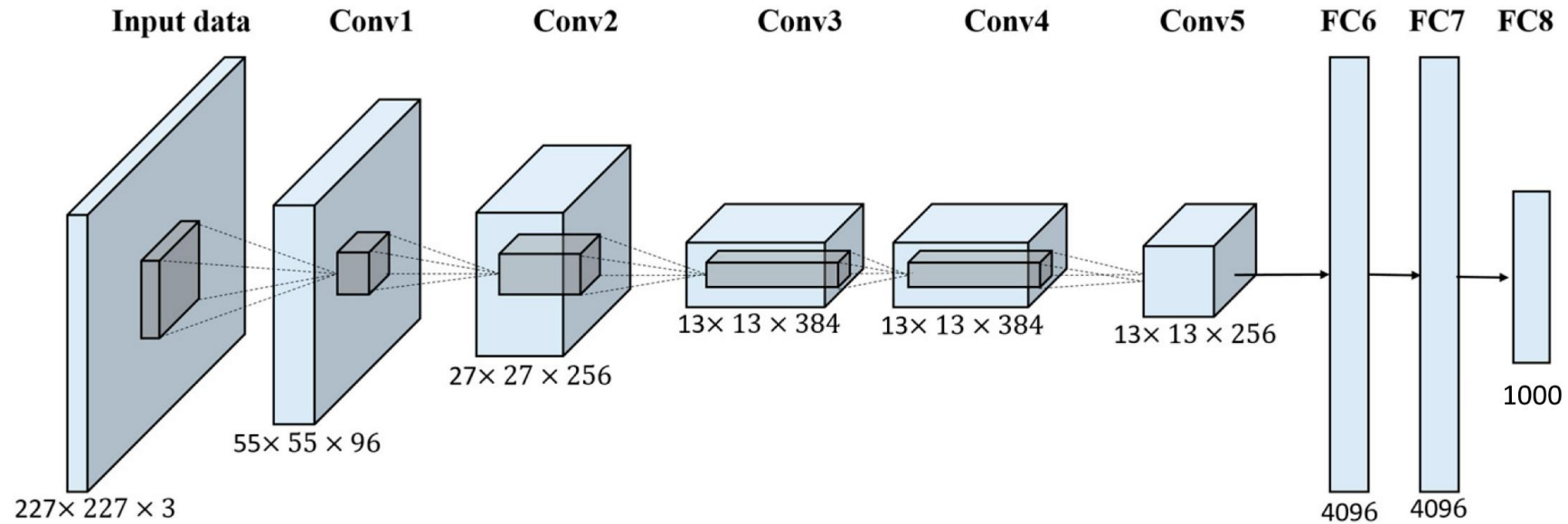
https://adamharley.com/nn_vis/

Cerremos con un caso de estudio

ImageNet Classification with Deep Convolutional Neural Networks

A. Krizhevsky, I. Sutskever, and G. E. Hinton (NeurIPS 2012)

AlexNet fue uno de los primeros ejemplos exitosos de ConvNets

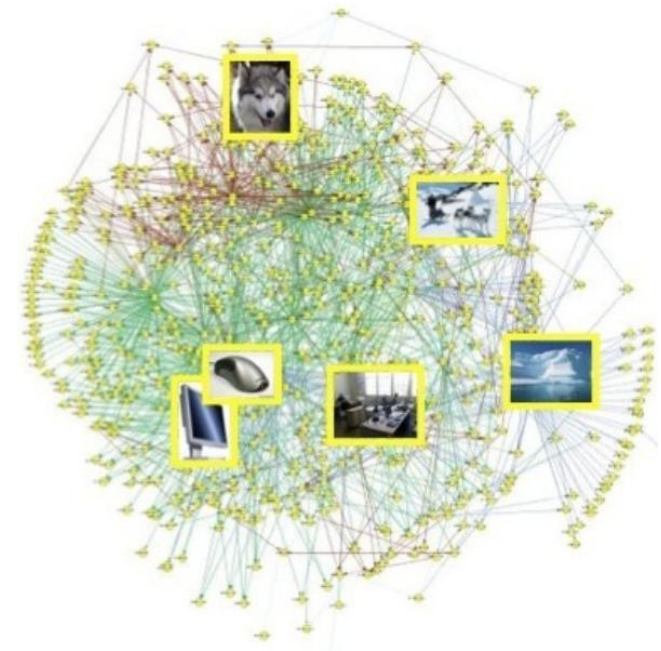


- Además de utilizar redes convoluciones, este trabajo introdujo una serie de “trucos” muy útiles para el entrenamiento de redes.
- Sin embargo, el gran diferenciador fue el ser capaz de aprender efectivamente de un set de datos órdenes de magnitud mayor que los existentes hasta ese momento.

Comencemos con los datos

- La base para la gran mayoría de los trabajos iniciales redes convolucionales para reconocimiento visual utiliza ImageNet.
- Este set de datos presenta más de 15M de imágenes divididas en 22K categorías.
- Además, imágenes están relacionadas por una estructura de grafo, que captura relaciones entre ellas.
- Las imágenes fueron recolectadas de la web y rotuladas utilizando Amazon Mechanical Turk.

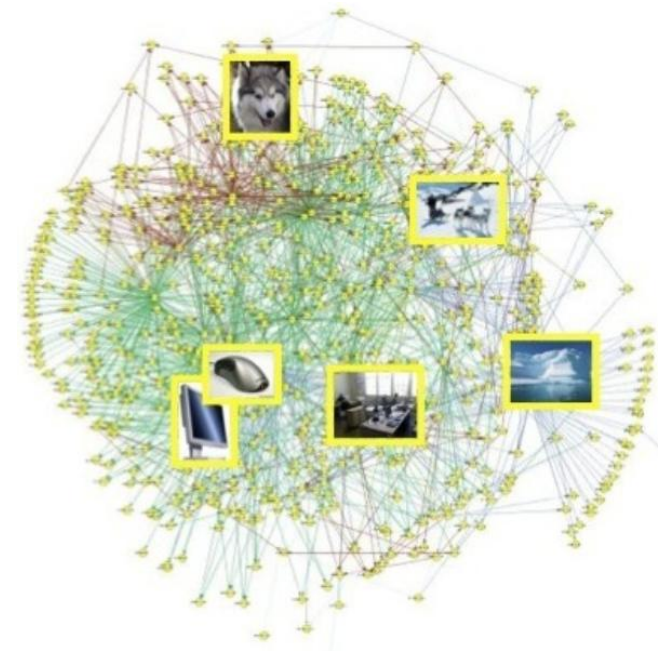
IMAGENET

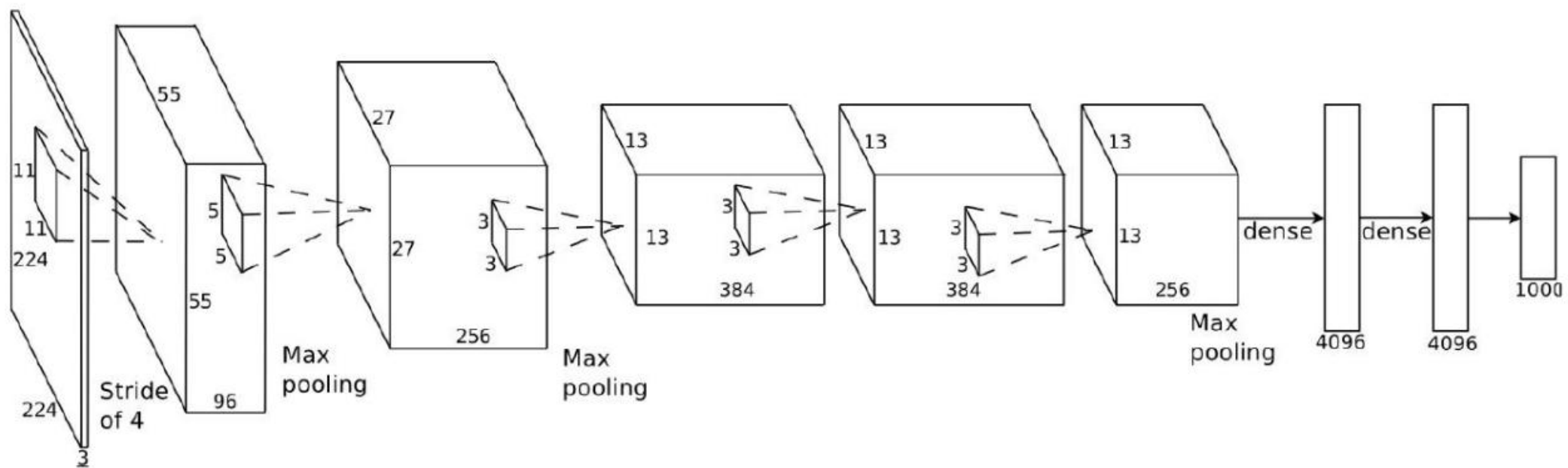


Comencemos con los datos

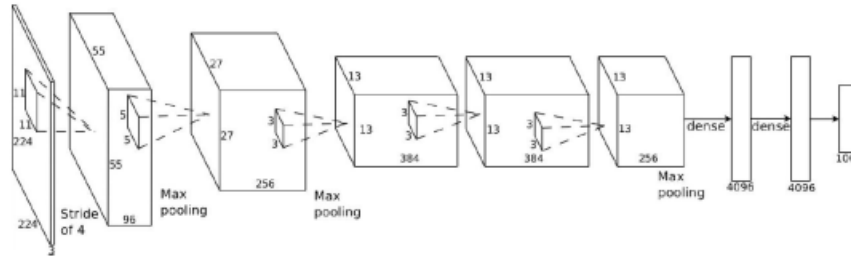
- Para realizar el entrenamiento, se utiliza un subconjunto de ImageNet, conocido como ILSVRC-XXXX (donde XXXX es el año).
- 1K categorías, 1.2M de imágenes de entrenamiento (1.2K por categoría).
- 50K imágenes de validación y 150K de test.
- Imágenes RGB de distintas resoluciones, pero escaladas a 256x256 para esta aplicación.
- Métricas de error top-1 y top-5.

IMAGENET





Input: $224 \times 224 \times 3 = 150528$ pixels



● Layer 1:

- nFilters = 96, Size = 11×11 , Depth = 3.
- nParams (weights) = $96 \times 11 \times 11 \times 3 = 34848$.
- Stride = 4, max-pooling = $3 \times 3:1$, nNeurons (outputs) = $55 \times 55 \times 96 = 290400$.
(obs: they use 3 zero-padding, then $(227 - 11)/4 + 1 = 55$)

● Layer 2:

- nFilters=256, Size = 5×5 , Depth = 96.
- nParams = $256 \times 5 \times 5 \times 96 = 614400$.
- Stride=1, max-pooling = $3 \times 3:2$, nNeurons= $27 \times 27 \times 256 = 186624$.

● Layer 3:

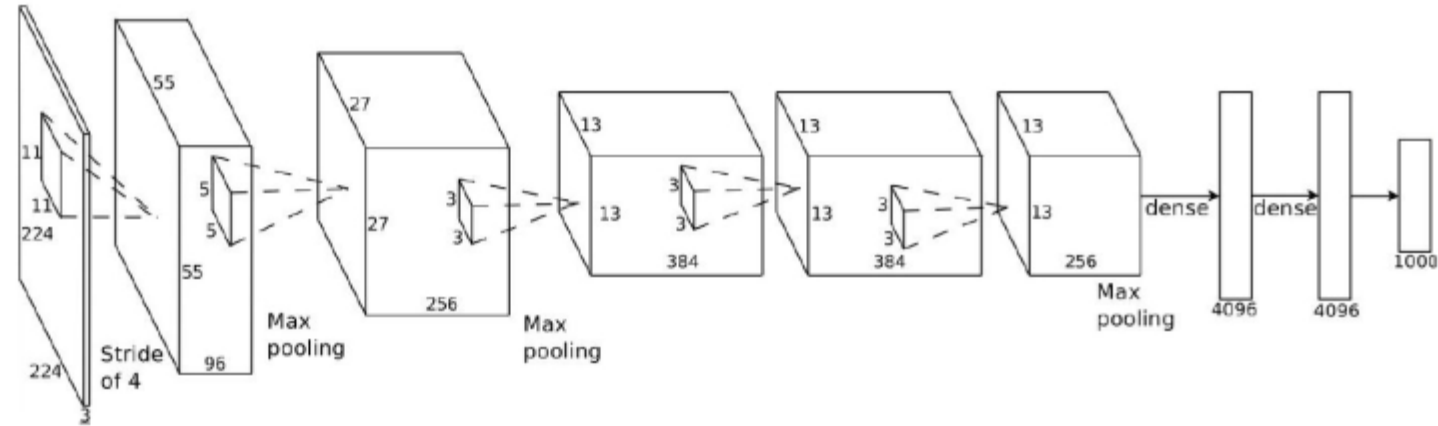
- nFilters = 384, Size = 3×3 , Depth = 256.
- nParams = $384 \times 3 \times 3 \times 256 = 884736$.
- Stride=1, max-pooling = none, nNeurons = $13 \times 13 \times 384 = 64896$.

● Layer 4:

- nFilters= 384, Size= 3×3 , Depth= 384.
- nParams = $384 \times 3 \times 3 \times 384 = 1327104$.
- Stride=1, max-pooling = none, nNeurons= $13 \times 13 \times 384 = 64896$.

● Layer 5:

- nFilters = 256, Size= 3×3 , Depth= 384.
- nParams = $256 \times 3 \times 3 \times 384 = 884736$.
- Stride=1, max-pooling = $3 \times 3:2$, nNeurons= $13 \times 13 \times 256 = 43264$.



- Layer 6: Fully connected to previous layer

- nParams = $6 \times 6 \times 256 \times 4096 = 37.748.736$.
- nNeurons = 4096.

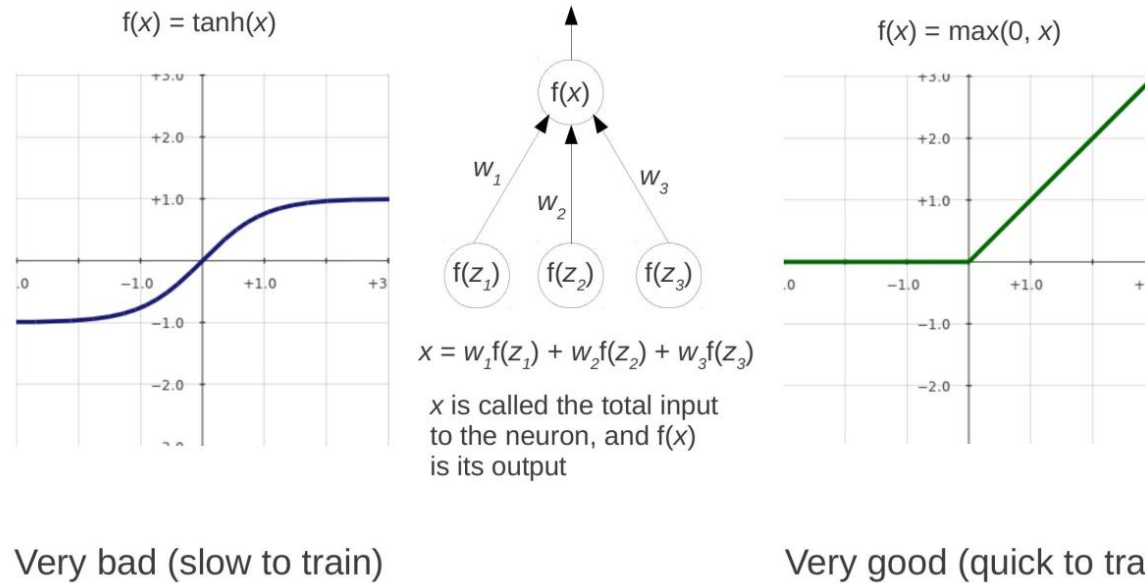
- Layer 7 : Fully connected to previous layer

- nParams = $4096 \times 4096 = 16.777.216$.
- nNeurons = 4096.

- Layer 8: Fully connected to previous layer

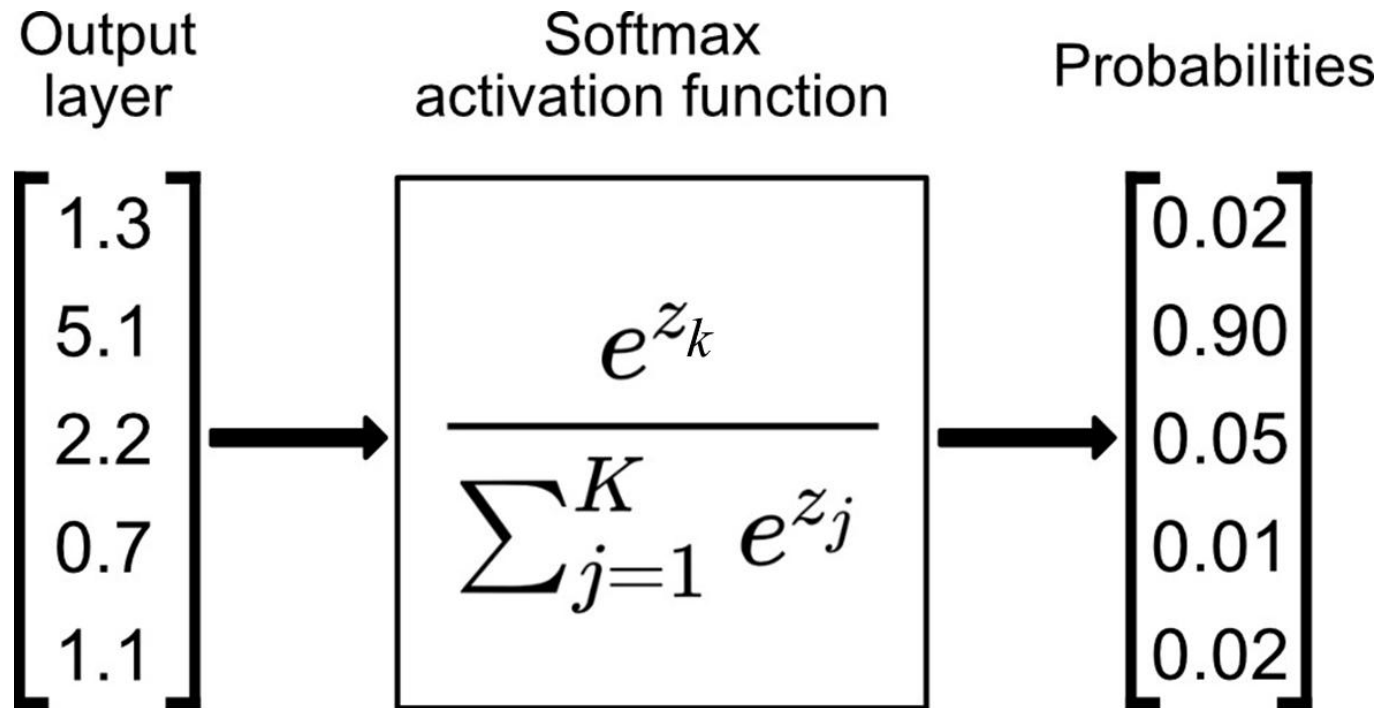
- nParams = $4096 \times 1000 = 4.096.000$.
- nNeurons = 1000.

Un aspecto central de su éxito fue el uso la función de activación **ReLU**



- AlexNet fue uno de los primeros trabajos en cambiar las no linealidades clásicas por ReLU.
- Esto permitió que el entrenamiento fuera varias veces más rápido.

Las salidas de AlexNet son las probabilidades de que la entrada x corresponda a cada una de las $K = 1000$ categorías posibles, $P(Y = k | X = x)$. Esto se logra utilizando la función de activación *softmax* luego de la última capa.



Como función de pérdida, se utilizó la función Cross Entropy, que combinada con la función *softmax*, genera una expresión bastante simple y compacta

Want to maximize the log likelihood, or (for a loss function) to minimize the negative log likelihood of the correct class:

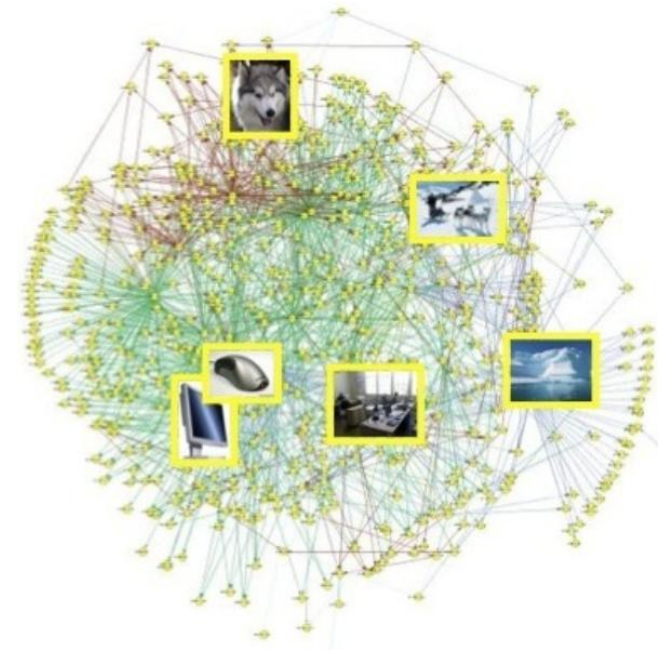
$$L_i = -\log P(Y = y_i | X = x_i)$$

in summary: $L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$

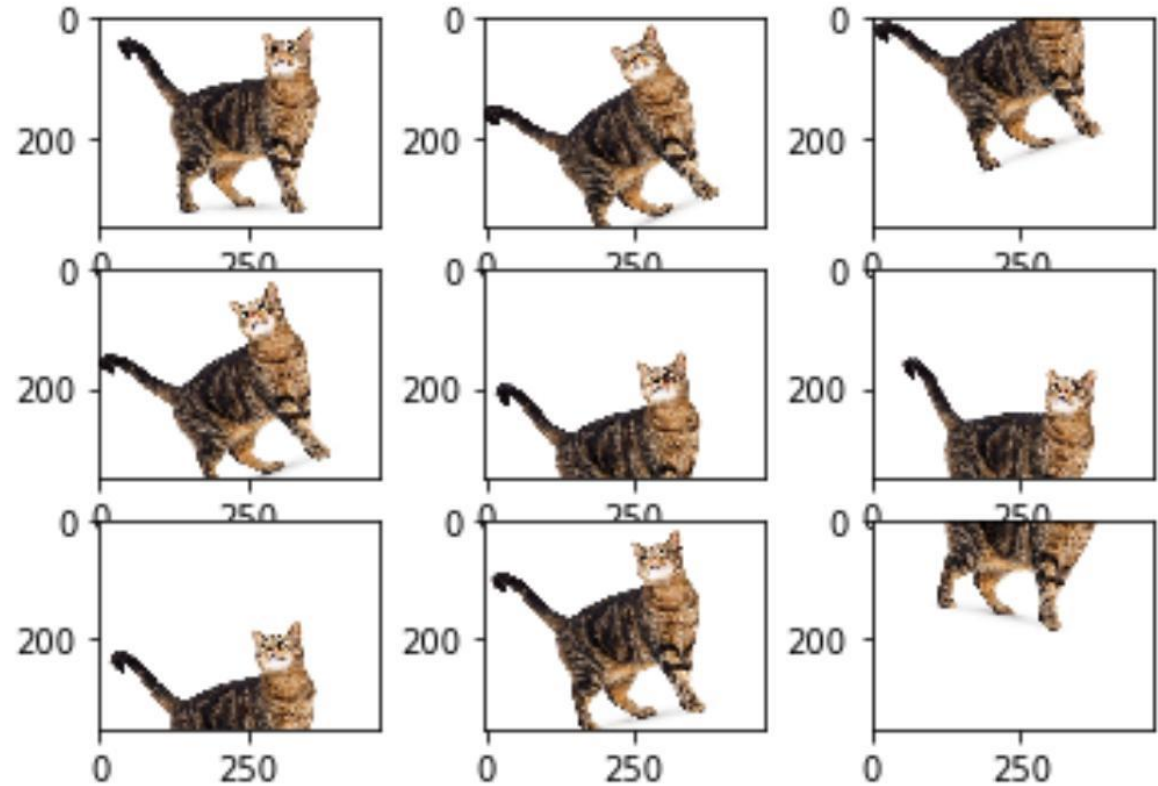
Otro aporte de AlexNet fue popularizar técnicas de **aumento de datos** para controlar el sobreajuste

- Si bien el set de datos es grande, el tamaño de la red obliga a aumentarlo aún más.
- Para esto se utilizaron técnicas de **aumento de datos**
- Por cada imagen se tomaron 5 subimágenes de 224x224 (4 esquinas y centro).
- Este proceso también se repite para el reflejo de cada imagen.
- Con esto, lograron aumentar 10 veces el tamaño efectivo del set de datos.
- Finalmente, a cada imagen se le resta la media de cada canal de color

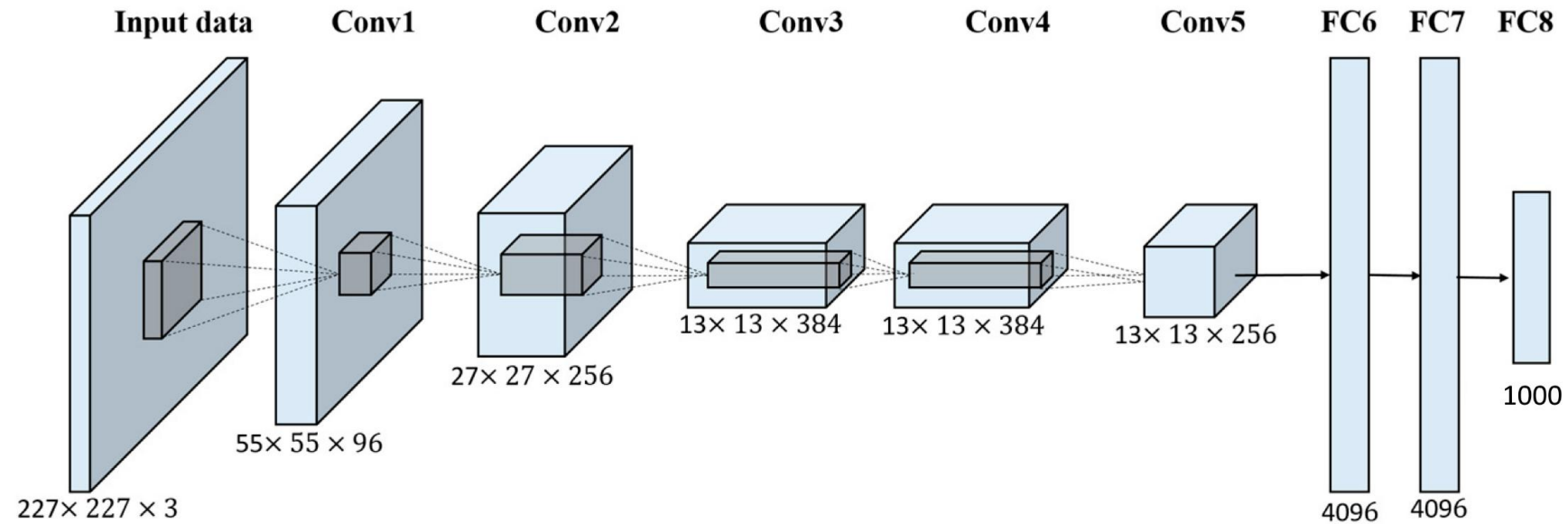
IMAGENET



Aumento de datos puede verse como la otra cara de la moneda de la Navaja de Occam => aumentamos la complejidad del problema, en vez de reducir la del modelo

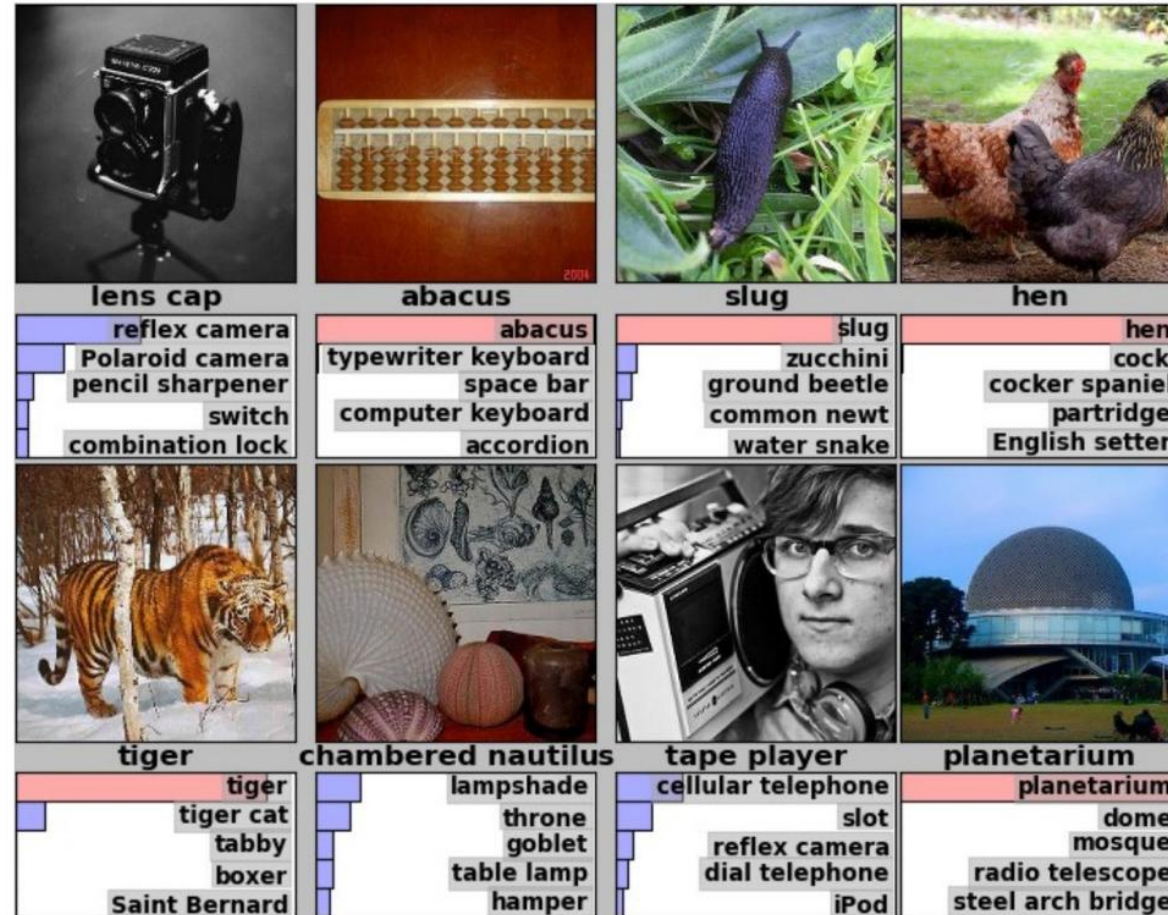


La combinación de todo esto generó resultados sorprendentes para la época

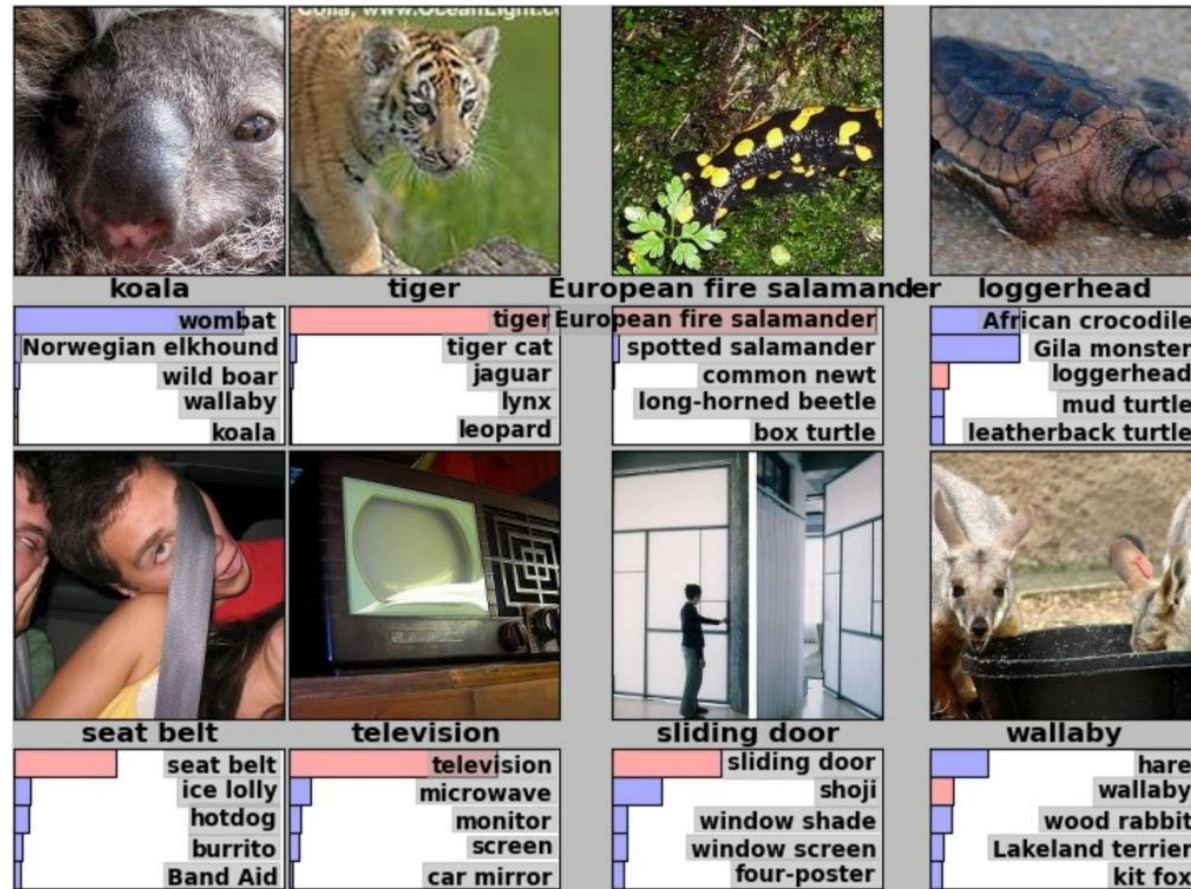


- Error top-5: 15.3%
- Error top-1: 26.2%
- Error humano: aprox. 8%

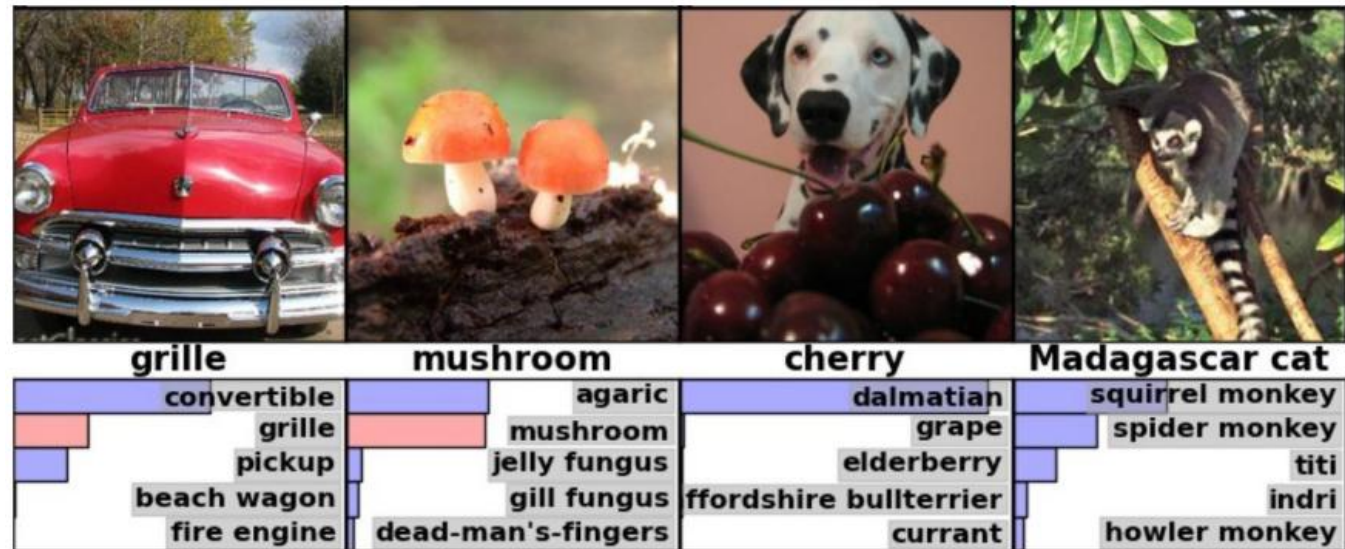
La combinación de todo esto generó resultados sorprendentes para la época



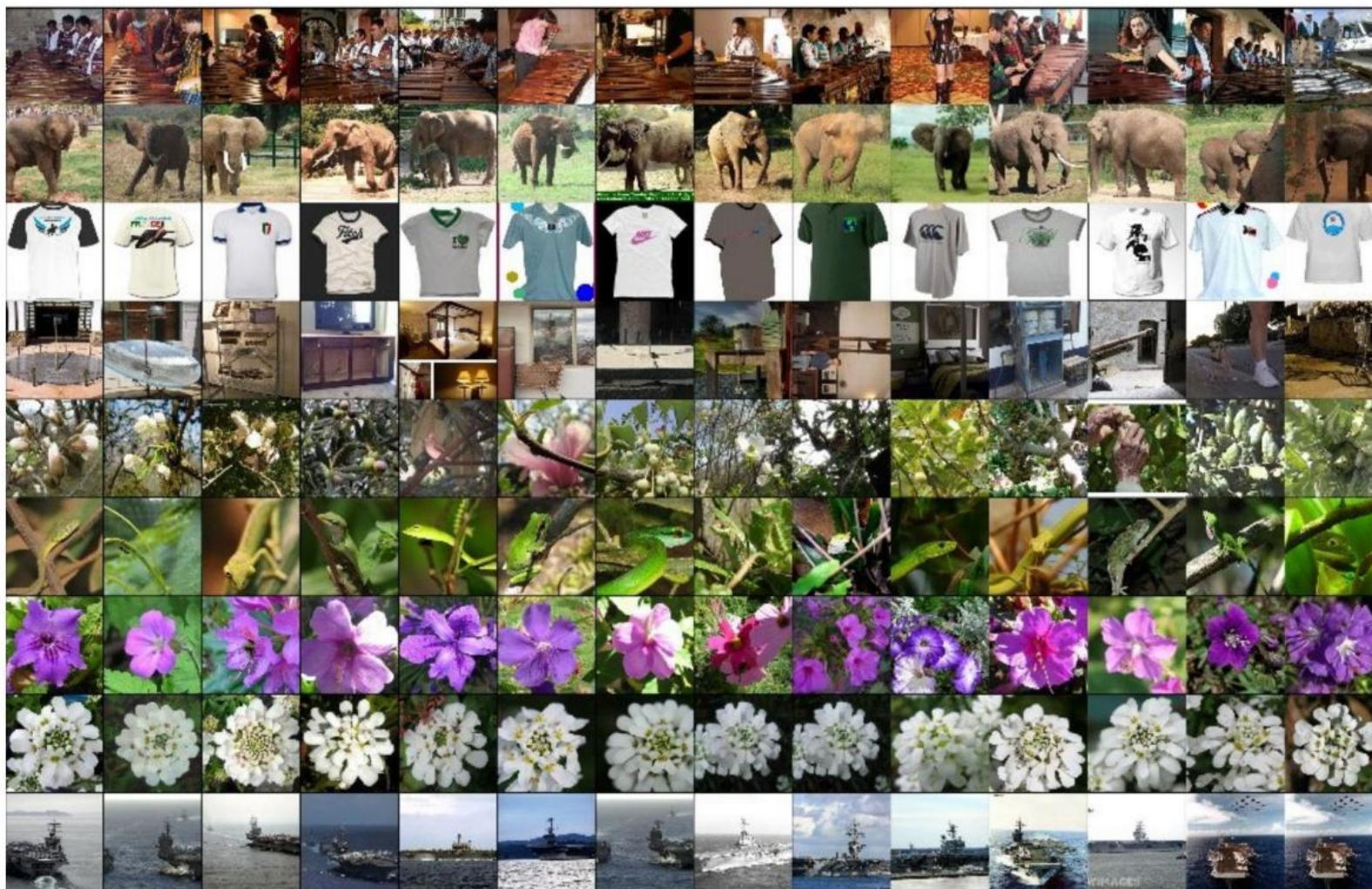
La combinación de todo esto generó resultados sorprendentes para la época



La combinación de todo esto generó resultados sorprendentes para la época



La combinación de todo esto generó resultados sorprendentes para la época



Rendimiento ha continuado mejorando



(a) Siberian husky



(b) Eskimo dog

- En la actualidad, las arquitecturas para reconocimiento visual son bastante más profundas (cientos de capas).
- Mejores métodos tienen alrededor de 89% top-1 y 99% top-5.
- Algunas muestras con código pueden verse en <https://paperswithcode.com/sota/image-classification-on-imagenet>

Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación



IIC2613 - Inteligencia Artificial

Redes Neuronales Convolucionales (CNN)

Hans Löbel

Dpto. Ingeniería de Transporte y Logística
Dpto. Ciencia de la Computación